



Secteur technique — Bachelier en Automatisation
Année académique 2025-2026

Projet final du 2^e quadrimestre

Système de contrôle automatique d'un ascenseur

Unités d'enseignement :

A208 : Acquisition et visualisation de données

A211 : Systèmes embarqués I

Étudiants : EL YAZAMI Yassine, TABICH Mohamed

Classe : 2AU

Professeurs : Emile COSTA, François DONNAY

Plateforme : Carte EasyPIC V7 — PIC18F45K22 8 MHz —
MikroC / LabVIEW

Année académique 2025-2026

Soumis le 09/06/2026

Table des matières

1. Introduction générale.....	3
2. Analyse fonctionnelle et cahier des charges	3
2.1 Contexte et scénario.....	3
2.2 Fonctions du système.....	3
2.3 Exigences et traçabilité.....	4
3. Partie Systèmes embarqués (A211).....	5
3.1 Schéma fonctionnel global (schéma blocs).....	5
3.2 Schéma électronique complet	5
3.3 Contextualisation logicielle	6
3.4 Présentation des algorithmes.....	7
3.5 Structure du programme.....	8
3.6 Gestion des interruptions.....	9
3.7 Description détaillée du protocole	9
Trames émises par le PIC vers LabVIEW	9
Trames émises par LabVIEW vers le PIC	10
3.8 Interactions avec le hardware	10
3.9 Conclusion partielle (A211)	11
4. Partie Acquisition & Visualisation (A208).....	12
4.1 Contextualisation logicielle	12
4.2 Présentation et explication de la face-avant	12
4.3 Diagramme de l'application avec analyse détaillée	13
Événement Timeout — réception et traitement des trames.....	13
Événements Connecter et Déconnecter.....	14
Événements d'appel d'étage.....	15
Événements de mode AUTO / MANU	15
Configuration des seuils (Envoyer config)	16
Commandes manuelles du moteur et des portes.....	16
Sécurité : urgence, acquittement, alarme et reset.....	18
Gestion de la mémoire EEPROM	19
Gestion de l'acquisition et de l'historique.....	20
4.4 Intégration avec le microcontrôleur	22
4.5 Traitement des données acquises	22
4.6 Conclusion partielle (A208)	22
5. Tests et résultats	23
6. Pistes d'amélioration	24
7. Conclusion générale.....	24
8. Références bibliographiques.....	25
9. Annexes — Code source complet.....	26

1. Introduction générale

Ce rapport présente la conception, la réalisation et la validation d'un système de commande automatique d'ascenseur, développé dans le cadre du projet pluridisciplinaire de fin de deuxième quadrimestre du Bachelier en Automatisation à la Haute École EPHEC Tech. Le projet s'appuie sur deux cours complémentaires : Systèmes embarqués I (A211), consacré à la programmation du microcontrôleur PIC18F45K22 en langage C, et Acquisition et visualisation de données (A208), dédié au développement d'une interface de supervision sous LabVIEW.

Le système reproduit le fonctionnement d'un ascenseur desservant quatre étages, de 0 à 3. Le cœur de la commande repose sur un PIC18F45K22 monté sur une carte EasyPIC V7, cadencé à 8 MHz et programmé avec MikroC. Le microcontrôleur lit l'état des capteurs, comme le poids de la cabine ou la présence à la porte, pilote le moteur de translation via un pont en H L293D commandé en PWM, affiche l'état du système sur un écran LCD 16×2 et stocke certaines données de fonctionnement dans une mémoire EEPROM externe reliée en I²C.

Une interface homme-machine développée sous LabVIEW permet de superviser le système depuis un PC. Elle communique avec le microcontrôleur en UART à 9600 bauds, à travers un protocole de trames délimitées par les caractères « < » et « > », conçu pour fonctionner dans les deux sens avec acquittement. L'opérateur peut suivre en direct la position de la cabine, la charge, le mode de fonctionnement et l'historique des trajets. Il peut aussi agir sur le système en appelant un étage, en passant du mode automatique au mode manuel, en déclenchant ou en acquittant une alarme, en lançant un arrêt d'urgence, en réglant les seuils de poids et de vitesse, ou encore en consultant et en réinitialisant la mémoire EEPROM.

Le document suit la structure demandée dans les consignes. Après une analyse fonctionnelle et un cahier des charges, il détaille la partie embarquée, avec l'architecture matérielle, les algorithmes, la structure logicielle, les interruptions, le protocole et les interactions matérielles, puis la partie acquisition et visualisation, avec la face avant, le diagramme, l'intégration et le traitement des données. Il se termine par les tests réalisés, des pistes d'amélioration, une conclusion générale, les références, ainsi qu'en annexe le code source complet.

2. Analyse fonctionnelle et cahier des charges

2.1 Contexte et scénario

Le scénario choisi est celui d'un ascenseur de bâtiment à quatre niveaux, surveillé depuis un poste de contrôle. La cabine se déplace d'un étage à l'autre à la demande, tout en respectant les règles de sécurité : elle ne démarre pas si la porte est ouverte ou si la charge dépasse la limite fixée, elle s'arrête tout de suite en cas d'arrêt d'urgence et reste bloquée en cas d'alarme jusqu'à son acquittement. Le fonctionnement peut être géré localement, en mode automatique par le PIC, ou à distance par l'opérateur, en mode manuel depuis LabVIEW.

2.2 Fonctions du système

Fonction principale FP1 — déplacer la cabine vers l'étage demandé. Le microcontrôleur choisit le sens de déplacement, ferme la porte, fait démarrer le moteur progressivement, gère le trajet entre les étages, puis ralentit et s'arrête à la destination.

Fonction principale FP2 — superviser et commander le système à distance. L'IHM LabVIEW reçoit en continu l'état du système et envoie les commandes de l'opérateur, avec acquittement.

Fonctions de contrainte — assurer la sécurité, mémoriser les données importantes et afficher l'état localement. Cela comprend l'arrêt d'urgence, l'alarme, la détection de surcharge, la détection de porte ouverte, la mémorisation du compteur de trajets, du dernier étage desservi et de la vitesse configurée, ainsi que l'affichage sur l'écran LCD.

2.3 Exigences et traçabilité

Le tableau ci-dessous reprend les exigences techniques imposées par l'énoncé et indique pour chacune sa mise en œuvre dans le projet.

Exigence (énoncé)	Mise en œuvre dans le projet
Au moins 3 entrées digitales (boutons-poussoirs)	Boutons d'appel d'étage RD0-RD3, acquittement RD4, urgence RB6, alarme RB7
Au moins 2 LED et une sortie PWM	LED1 verte (RA2) porte/disponibilité, LED2 rouge (RA3) défaut ; PWM moteur sur CCP1/RC2
2 entrées analogiques	Potentiomètre de poids sur AN1/RA1 (la voie AN0 reste disponible) acquises par ADC
UART bidirectionnel (duplex) avec réception sous interruption	EUSART1 à 9600 bauds, émission par scrutation, réception sur interruption RC1IF
Afficheur LCD 16×2 sur PORTB	LCD en mode 4 bits : données RB0-RB3, RS=RB4, EN=RB5
Trois interruptions simultanées	Timer0 (16 bits), interruption sur changement d'état RB6/RB7, réception UART
Mémoire EEPROM via I ² C (lecture/écriture)	EEPROM NM24C08 sur MSSP1 I ² C (SCL=RC3, SDA=RC4)
Application LabVIEW modulaire avec sous-VI et sauvegarde fichier	Architecture événementielle, sous-VI VISA/affichage/sauvegarde/alarmes, export fichier .txt
Protocole bidirectionnel avec acquittements et gestion d'erreurs	Trames < > ; réponses <ACK,OK> / <ACK,ERR>

Contraintes de réalisation — respect du brochage de la carte EasyPIC V7, développement sous MikroC, validation en simulation Proteus et sur carte physique, microcontrôleur cadencé à 8 MHz.

3. Partie Systèmes embarqués (A211)

3.1 Schéma fonctionnel global (schéma blocs)

Le schéma fonctionnel ci-dessous montre comment le système est organisé. Le **PIC** est au centre et regroupe cinq parties du programme : la gestion des interruptions (base de temps, urgence, réception série), la lecture des capteurs (poids via l'ADC, porte), la logique de commande (modes, appels, gestion des étages), la commande du moteur (PWM et rampes) et, enfin, l'affichage ainsi que l'envoi des données (LCD et trames série).

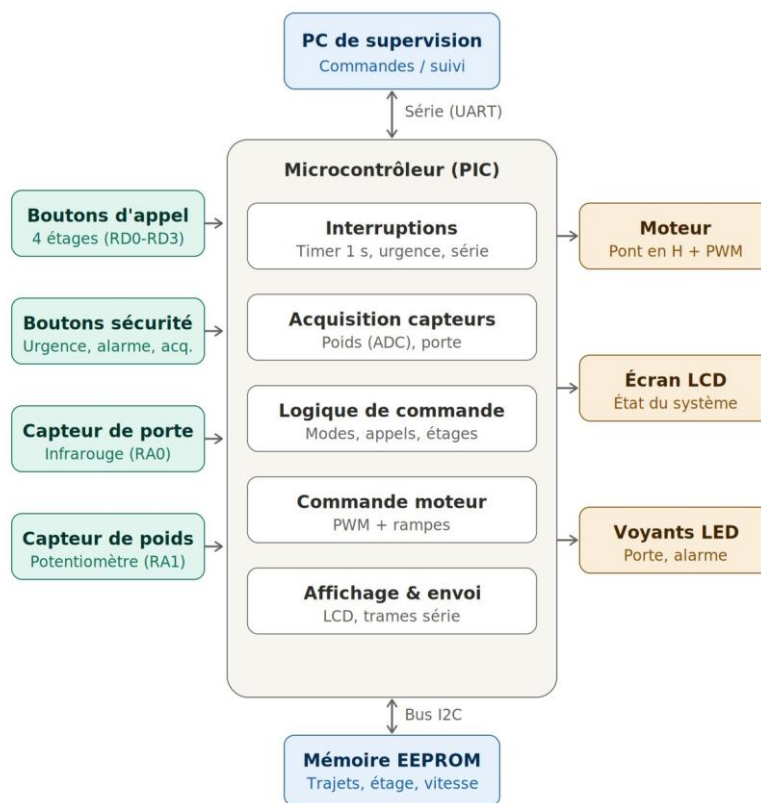


Figure 1 — Schéma fonctionnel global du système.

À gauche se trouvent les entrées du système : les boutons d'appel des quatre étages (**RD0 à RD3**), les boutons de sécurité (**urgence, alarme et acquittement**), le capteur de porte infrarouge (**RA0**) et le capteur de poids, représenté par un potentiomètre sur **RA1**.

À droite se trouvent les éléments pilotés par le système : le moteur commandé par le pont en H et la **PWM**, l'écran **LCD** qui affiche l'état du système et les voyants **LED**.

Dans la partie supérieure, le PC de supervision communique avec le **PIC** via une liaison série **UART**. Dans la partie inférieure, la mémoire **EEPROM** est connectée par le bus **I²C** et conserve le compteur de trajets, le dernier étage desservi et la vitesse configurée.

3.2 Schéma électronique complet

Le schéma électronique, réalisé sous Proteus, regroupe l'ensemble des composants et de leurs raccordements au microcontrôleur.

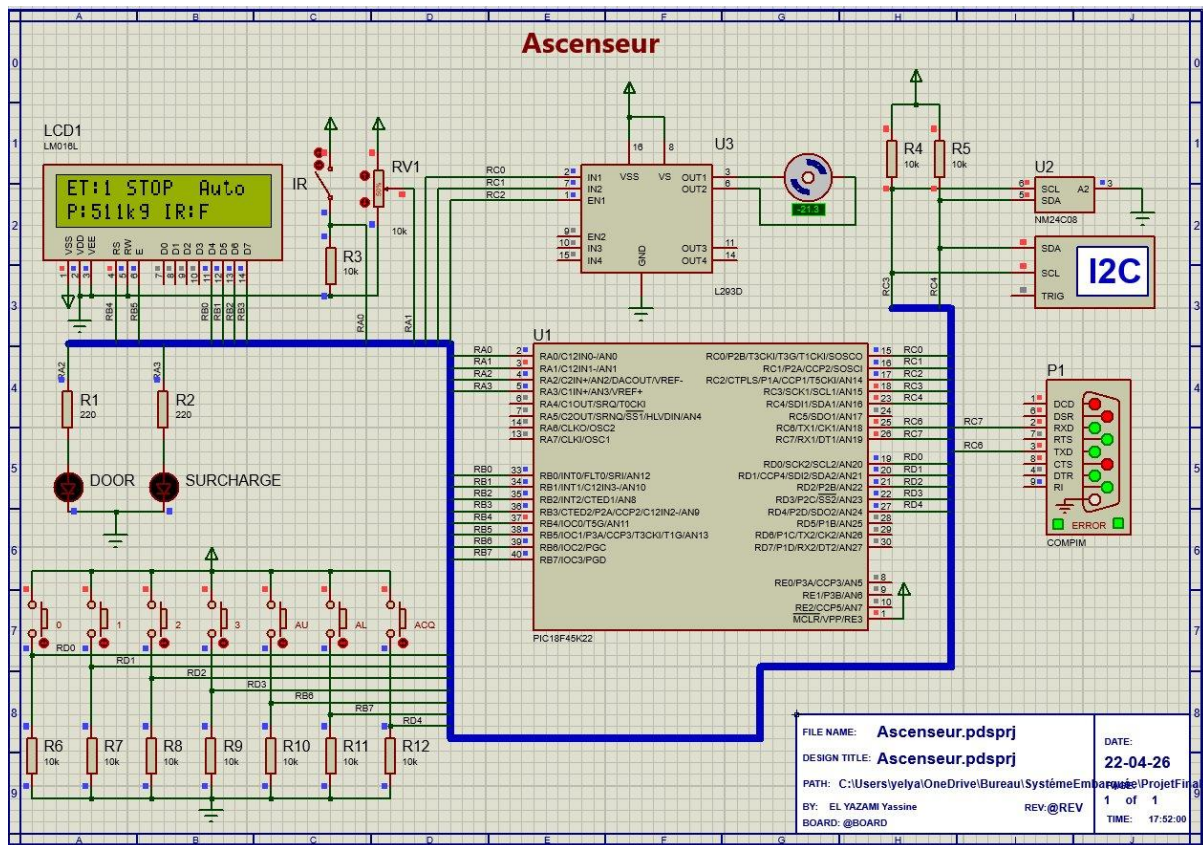


Figure 2 — Schéma électronique complet (Proteus).

Le **PIC18F45K22 (U1)** est au centre du montage et tout le reste est branché autour de lui. L'écran **LCD LM016L** est piloté en mode 4 bits par le **PORTB** : les lignes de données **D4 à D7** vont sur **RB0 à RB3**, le signal **RS** sur **RB4** et le signal **Enable (E)** sur **RB5**. Un potentiomètre de **10 kΩ** sert à régler le contraste.

Le pont en H **L293D (U3)** reçoit les ordres de sens sur **IN1 (RC0)** et **IN2 (RC1)**, ainsi que le signal d'activation venant de la sortie **PWM CCP1 (RC2)**. Ses sorties **OUT1** et **OUT2** commandent le moteur à courant continu.

La mémoire **EEPROM NM24C08 (U2)** est reliée au bus **I²C**, avec **SCL** sur **RC3** et **SDA** sur **RC4**. Deux résistances de tirage de **10 kΩ (R4 et R5)** assurent le bon fonctionnement du bus. La communication série passe par un port virtuel **COMPIM (P1)** relié à **RC6 (TX)** et **RC7 (RX)**.

Du côté des entrées, le potentiomètre **RV1** simule la charge de la cabine sur l'entrée analogique **RA1**, tandis qu'un contact simule le capteur de porte infrarouge sur **RA0**. Les sept boutons-poussoirs — appels **0 à 3**, urgence **AU**, alarme **AL** et acquittement **ACQ** — sont branchés sur **RD0 à RD3**, **RB6**, **RB7** et **RD4**, avec des résistances de rappel à la masse (**pull-down**) de **10 kΩ**.

Enfin, deux LED de signalisation, **DOOR** sur **RA2** et **SURCHARGE** sur **RA3**, sont protégées par des résistances de **220 Ω**.

3.3 Contextualisation logicielle

Le firmware est écrit en **C** avec **MikroC PRO for PIC**, qui fournit les bibliothèques utilisées pour le matériel : **ADC** pour lire les valeurs analogiques, **PWM (CCP1)** pour commander le moteur, **I²C (MSSP1)** pour la mémoire, **UART (EUSART1)** pour la liaison série et **Lcd** pour l'afficheur. Le programme exploite donc une grande partie des périphériques du **PIC18F45K22**.

Le logiciel s'appuie sur trois points principaux. D'abord, des interruptions très courtes qui servent surtout à poser des indicateurs, comme la base de temps ou l'urgence/alarme, ou à regrouper les caractères reçus pour former des trames complètes stockées dans une file circulaire. Ensuite, une boucle principale dans **main()** qui traite les événements reçus, exécute les commandes, met à jour l'affichage et, en mode

automatique, lance les déplacements. Enfin, des fonctions de déplacement qui restent surveillées : pendant les pauses, la fonction **attendre_ms()** continue de vérifier le bouton d'urgence, de traiter les commandes série et de rafraîchir régulièrement l'affichage ainsi que les trames de télémétrie. Le système reste donc réactif même pendant un mouvement.

Pour simplifier les tests, deux durées de temporisation sont prévues, selon la directive **SIMULATION_PROTEUS** : une version raccourcie pour aller plus vite dans Proteus, et une version normale pour la carte réelle. Avec ce seul changement de constantes, on peut utiliser exactement le même code dans les deux cas.

3.4 Présentation des algorithmes

Deux algorithmes décrivent le comportement du firmware : la boucle principale et le sous-programme de déplacement.

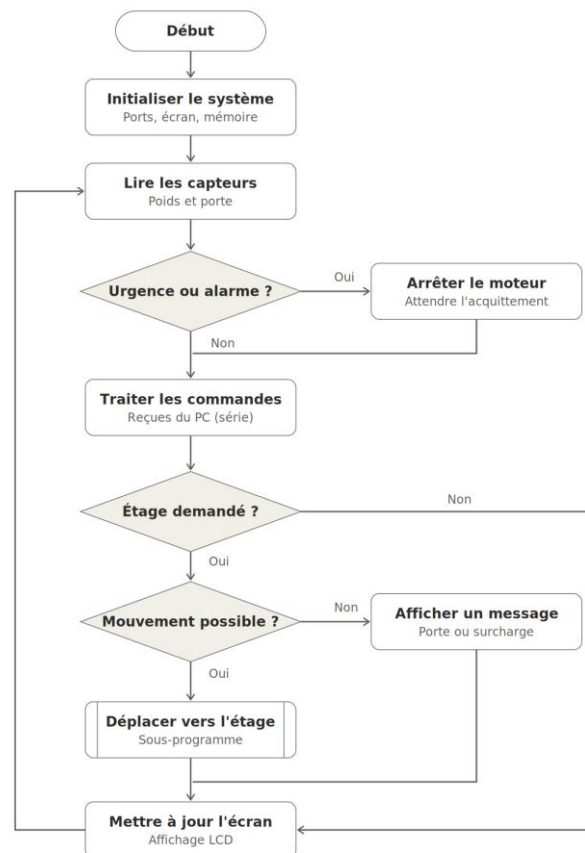


Figure 3 — Algorithme de la boucle principale.

La boucle principale commence par initialiser le système (ports, écran, mémoire, etc.), puis répète en continu les mêmes étapes. Elle lit d'abord l'état des capteurs, comme le poids et la porte, puis vérifie s'il y a une situation d'urgence ou une alarme. Dans ce cas, le moteur est arrêté et le système attend un acquiescement. Ensuite, les commandes reçues du PC sont traitées. Si un étage a été demandé et que les conditions sont réunies (porte fermée et absence de surcharge), le programme lance le déplacement de la cabine. À chaque cycle, l'affichage du LCD est également mis à jour.

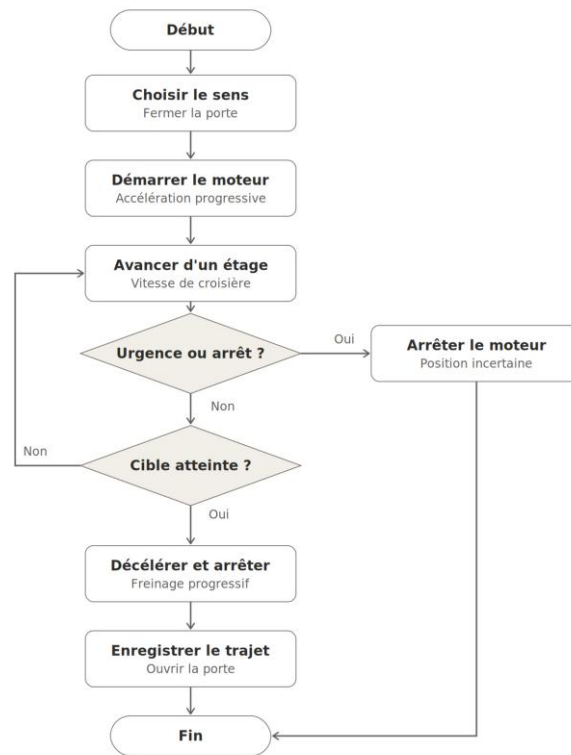


Figure 4 — Algorithme du sous-programme de déplacement.

Le déplacement commence par choisir le sens de marche et fermer la porte. Le moteur démarre ensuite progressivement avec une rampe d'accélération, puis la cabine avance étage par étage à vitesse normale. À chaque étape, le programme vérifie qu'aucune urgence ou demande d'arrêt n'est apparue ; si c'est le cas, le moteur est coupé et la position devient inconnue. Une fois l'étage demandé atteint, le moteur ralentit progressivement, le trajet est enregistré dans l'EEPROM et la porte s'ouvre. Cette logique correspond directement aux fonctions **deplacer_vers()**, **demarrer_moteur()**, **rampe_accel()** et **rampe_decel()** du code.

3.5 Structure du programme

Le programme est construit autour de plusieurs variables d'état globales et de différentes fonctions spécialisées. Les principales variables utilisées sont présentées ci-dessous.

Variable	Rôle
etage_actuel / etage_cible	Position courante et destination de la cabine
direction / en_mouvement	Sens ('U','D','S') et indicateur de cabine en mouvement
req[NB_ETAGES]	Tableau des appels d'étage en attente
poids_kg / seuil_surge / surcharge_active	Charge mesurée, seuil de surcharge et état de surcharge
mode_auto	Mode automatique (1) ou manuel (0)
urg_active / al_active / position_inconnue	États de sécurité : urgence, alarme, position perdue
nb_session / nb_trajets / last_etage	Compteurs de trajets (RAM/EEPROM) et dernier étage
vitesse_max_pc / pwm_max_eff / pwm_actuel	Vitesse en % , PWM maximale effective et PWM courante
cmd_queue / cmd_qhead / cmd_qtail	File circulaire des commandes reçues par UART

Les fonctions sont organisées en plusieurs groupes qui ont chacun leur rôle : les fonctions de bas niveau pour l'EEPROM, la PWM et la lecture des capteurs, celles qui gèrent l'affichage, celles qui s'occupent des échanges série et des acquittements, celles qui contrôlent les déplacements du moteur, et enfin celle qui analyse les commandes reçues. La fonction **main()** prend ensuite le relais pour faire fonctionner l'ensemble après l'initialisation des périphériques.

3.6 Gestion des interruptions

Conformément à l'énoncé, le système gère trois sources d'interruption simultanées, traitées dans une routine unique `interrupt()`.

Source	Déclenchement	Traitement
Timer0 (16 bits)	Débordement périodique (≈ 1 s, prédiviseur 1:256, rechargement TMR0H/L)	Positionne <code>timer0_flag</code> et incrémente <code>timer0_count</code> ; provoque, dans la boucle, la lecture des capteurs, la mise à jour du LCD et l'envoi de la trame DATA
Changement d'état RB6 / RB7	Appui sur le bouton d'urgence (RB6) ou d'alarme (RB7)	Coupe immédiatement le moteur, allume la LED2 et arme le drapeau d'urgence (<code>urgence_flag</code>) ou d'alarme (<code>al_flag</code>)
Réception UART (RC1IF)	Arrivée d'un caractère sur RC7	Assemble les caractères entre ' <code><</code> ' et ' <code>></code> ' dans <code>rx_buf</code> , gère le débordement de réception (OERR), puis dépose la trame complète dans la file circulaire <code>cmd_queue</code>

Le choix de garder des interruptions très courtes est volontaire : la routine ne fait aucun traitement lourd, comme écrire sur le LCD ou envoyer des données en série. L'arrêt d'urgence est aussi vérifié dans la boucle principale et dans `attendre_ms()`, avec un contrôle anti-rebond, pour garantir l'arrêt du moteur même pendant une temporisation. La file circulaire de commandes permet enfin de séparer la réception des trames, qui se fait de manière asynchrone sous interruption, de leur exécution, qui se fait dans la boucle principale. Cela évite de perdre des trames quand un traitement prend du temps.

3.7 Description détaillée du protocole

La communication se fait en série asynchrone à **9600 bauds**, au format **8N1**. Chaque message est envoyé sous forme d'une trame **ASCII**, encadrée par « `<` » au début et « `>` » à la fin, ce qui facilite la reprise de la réception en cas d'erreur. Le protocole fonctionne dans les deux sens et il est accompagné d'un acquittement : à chaque commande reçue, le microcontrôleur renvoie une réponse pour confirmer si tout s'est bien passé ou non.

Trames émises par le PIC vers LabVIEW

Trame	Description
<code><DATA,ET:e,DIR:d,PT:p,PRT:r,AL:a,URG:u,NB:n,PWM:w,TPS:t></code>	Télémétrie périodique de l'état du système, émise sur base de temps Timer0 et lors des transitions
<code><EEP,TRAJ:x,LAST:y,VITESSE:z></code>	Réponse à la commande GET:EEP : contenu de la mémoire EEPROM
<code><ACK,OK></code>	Acquittement positif : commande acceptée et exécutée
<code><ACK,ERR></code>	Acquittement négatif : commande refusée (conditions non remplies)

La signification des champs de la trame DATA est précisée ci-dessous. On notera que la direction et l'état de porte sont transmis sous forme numérique, ce qui simplifie l'analyse côté LabVIEW.

Champ	Signification	Valeurs
ET	Étage actuel	0 à 3
DIR	Direction	0 = arrêt, 1 = montée, 2 = descente
PT	Poids de la cabine (kg)	valeur numérique
PRT	État de la porte	1 = ouverte, 0 = fermée
AL	Alarme active	0 / 1
URG	Arrêt d'urgence actif	0 / 1
NB	Compteur de trajets de la session (RAM)	valeur numérique
PWM	Rapport cyclique moteur (%)	0 à 100
TPS	Durée du dernier trajet (s, mesurée par Timer0)	valeur numérique

Trames émises par LabVIEW vers le PIC

Commande	Description	Mode
<CMD,CALL:X>	Appel de la cabine à l'étage X	Auto
<CMD,STOP>	Arrêt d'urgence depuis le PC	Les deux
<CMD,ACK>	Acquittement / vérification de l'état système	Les deux
<CMD,MODE:AUTO>	Bascule en mode automatique	Les deux
<CMD,MODE:MAN>	Bascule en mode manuel	Les deux
<CMD,PMAX:X>	Réglage du poids maximal (kg)	Les deux
<CMD,SPD:X>	Réglage de la vitesse moteur (%)	Manuel
<CMD,DOOR:O> / <CMD,DOOR:F>	Ouverture / fermeture manuelle de la porte	Manuel
<CMD,MOT:UP> / <CMD,MOT:DWN>	Montée / descente d'un étage	Manuel
<CMD,MOT:STOP>	Arrêt propre du moteur (sans alarme)	Manuel
<CMD,AL:ON>	Déclenchement de l'alarme depuis le PC	Les deux
<CMD,RST:AL>	Réinitialisation de l'alarme	Les deux
<CMD,GET:EEP>	Lecture des données EEPROM	Les deux
<CMD,EEP:RST>	Remise à zéro des données EEPROM	Les deux

La communication fonctionne donc bien dans les deux sens : le PC reçoit en permanence l'état du système et peut envoyer des commandes dont l'exécution est confirmée par le microcontrôleur. En cas de problème, celui-ci renvoie un message **<ACK,ERR>** lorsque la commande ne peut pas être exécutée, par exemple si un étage est demandé alors que la cabine est en surcharge, ou si une commande manuelle est envoyée alors que le système est en mode automatique.

3.8 Interactions avec le hardware

Le tableau suivant récapitule l'affectation des broches et leur rôle, conformément au brochage de la carte EasyPIC V7.

Broche	Signal	Sens	Fonction
RA0	IR porte	Entrée num.	Capteur de porte (appuyé = porte ouverte)
RA1/AN1	Poids	Entrée ADC	Potentiomètre simulant la charge 0-900 kg

Broche	Signal	Sens	Fonction
RA2 / RA3	LED1 / LED2	Sorties	Porte/disponibilité (verte) ; défaut/surcharge (rouge)
RB0-RB3	LCD D4-D7	Sorties	Bus de données LCD (mode 4 bits)
RB4 / RB5	LCD RS / EN	Sorties	Register Select et Enable du LCD
RB6 / RB7	Urgence / Alarme	Entrées IOC	Interruptions sur changement d'état
RC0 / RC1	L293D IN1 / IN2	Sorties	Sens de rotation du moteur
RC2/CCP1	L293D EN	Sortie PWM	Activation et vitesse du moteur
RC3 / RC4	SCL / SDA	Sortie / E-S	Bus I ² C vers l'EEPROM
RC6 / RC7	UART TX / RX	Sortie / Entrée	Liaison série 9600 bauds
RD0-RD3	Appels 0-3	Entrées	Boutons d'appel d'étage (pull-down)
RD4	ACQ	Entrée	Bouton d'acquiescement (pull-down)

La commande du moteur suit le fonctionnement du **L293D** : lorsque le signal **EN** vaut 0, le moteur est arrêté. Quand **EN** est actif, la combinaison **IN1 = 1 / IN2 = 0** fait monter la cabine, tandis que **IN1 = 0 / IN2 = 1** la fait descendre. La vitesse est réglée par le rapport cyclique de la **PWM**, calculé à partir de la vitesse demandée en pourcentage.

La mesure du poids est rendue plus fiable en faisant la moyenne de huit lectures successives de l'ADC, puis en convertissant le résultat sur une plage de **0 à 900 kg**. L'EEPROM communique en **I²C** à **100 kHz** avec les octets de contrôle **0xA0** en écriture et **0xA1** en lecture. La **PWM** fonctionne à **5 kHz** et la liaison série à **9600 bauds**. Tous ces paramètres sont configurés lors de l'initialisation du programme dans la fonction **main()**.

3.9 Conclusion partielle (A211)

La partie embarquée répond à toutes les exigences du projet : gestion de trois interruptions en parallèle, communication **UART** dans les deux sens avec réception sous interruption, deux entrées analogiques, des sorties **LED** et **PWM**, un afficheur **LCD** ainsi qu'une mémoire **EEPROM** en **I²C**.

L'architecture choisie, basée sur des interruptions courtes, une file de commandes et des fonctions de déplacement surveillées, permet d'obtenir un système réactif et sûr. Les situations particulières, comme un arrêt d'urgence pendant un déplacement, une position devenue inconnue ou une surcharge, sont prises en compte de manière explicite.

Enfin, le même code source fonctionne aussi bien sous **Proteus** que sur la carte réelle, ce qui a facilité les tests et permis une mise au point progressive tout au long du développement.

4. Partie Acquisition & Visualisation (A208)

4.1 Contextualisation logicielle

L'interface de supervision a été développée sous LabVIEW. Elle repose sur une architecture événementielle basée sur une **Event Structure** placée dans une boucle **While**. Cette structure réagit aux actions de l'opérateur, comme l'appui sur un bouton ou la modification d'une valeur. Un événement de type **Timeout** est également utilisé pour lire régulièrement les données reçues sur le port série.

L'état de l'application est conservé d'une itération à l'autre grâce à des registres à décalage (**shift registers**), visibles dans le diagramme sous la forme des différentes mémorisations utilisées par le programme.

La communication série est assurée à l'aide des fonctions **VISA**, qui permettent l'ouverture, la configuration, la lecture, l'écriture et la fermeture du port de communication.

Conformément aux consignes du projet, l'application a été organisée de manière modulaire à l'aide de plusieurs sous-VI : un sous-VI dédié à la communication VISA, un sous-VI pour l'affichage des graphiques, un sous-VI pour l'enregistrement des données dans un fichier texte et un sous-VI pour la gestion des alarmes. Cette organisation rend le programme plus clair, plus facile à comprendre et plus simple à faire évoluer.

4.2 Présentation et explication de la face-avant

La face-avant rassemble, dans une disposition logique, les commandes et les indicateurs du système. La figure suivante en montre l'état initial, port fermé.

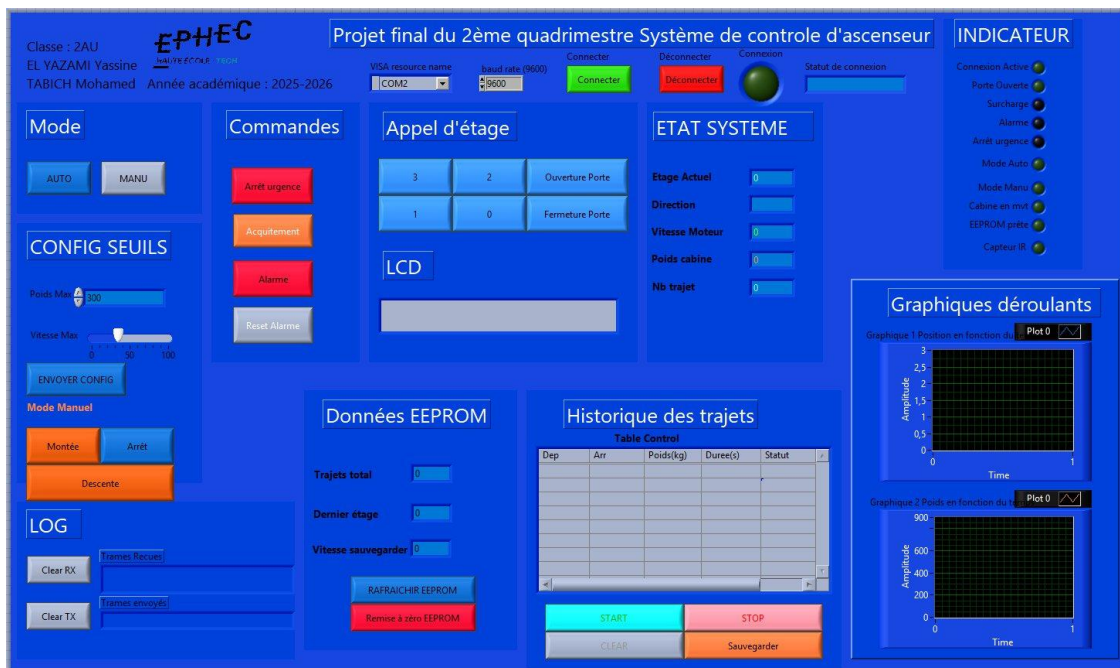


Figure 5 — Face-avant LabVIEW (état déconnecté).

Dans la partie supérieure de l'interface se trouvent les éléments liés à la connexion série : sélection de la ressource VISA, réglage du débit de communication, boutons **Connecter** et **Déconnecter**, ainsi qu'un voyant et un indicateur d'état de la connexion. Le titre du projet est également affiché dans cette zone.

La colonne de gauche regroupe les commandes de configuration du système : choix du mode **AUTO** ou **MANU**, réglage des seuils de poids maximal et de vitesse maximale, ainsi que le bouton **Envoyer config**. On y retrouve également les commandes manuelles de montée, de descente et d'arrêt, ainsi qu'un journal des trames échangées (**LOG**) accompagné des boutons **Clear RX** et **Clear TX**.

La colonne centrale rassemble les principales commandes de pilotage : arrêt d'urgence, acquittement, alarme, remise à zéro de l'alarme, appels des étages de 0 à 3, ainsi que les commandes d'ouverture et de fermeture de la porte. Une simulation de l'afficheur LCD du système y est également intégrée.

La colonne de droite affiche les informations de fonctionnement du système, comme l'étage actuel, la direction, la vitesse, le poids et le nombre de trajets effectués. Elle comprend aussi plusieurs indicateurs permettant de visualiser rapidement l'état du système : connexion active, porte ouverte, surcharge, alarme, arrêt d'urgence, mode automatique ou manuel, cabine en mouvement, EEPROM prête et état du capteur infrarouge.

Dans la partie inférieure se trouvent les informations enregistrées dans l'EEPROM, notamment le nombre total de trajets, le dernier étage desservi et la vitesse sauvegardée, avec les boutons **Rafraîchir** et **Remise à zéro**. Cette zone contient également le tableau d'historique des trajets ainsi que deux graphiques montrant l'évolution de la position de la cabine et du poids au cours du temps.

La figure suivante présente cette même interface en fonctionnement avec la liaison série active. On peut y voir la cabine positionnée à l'étage 3, un historique de trajets déjà rempli, un compteur indiquant 11 trajets effectués, la simulation du LCD qui reproduit l'affichage embarqué ainsi que les graphiques affichant l'évolution de la position et du poids.

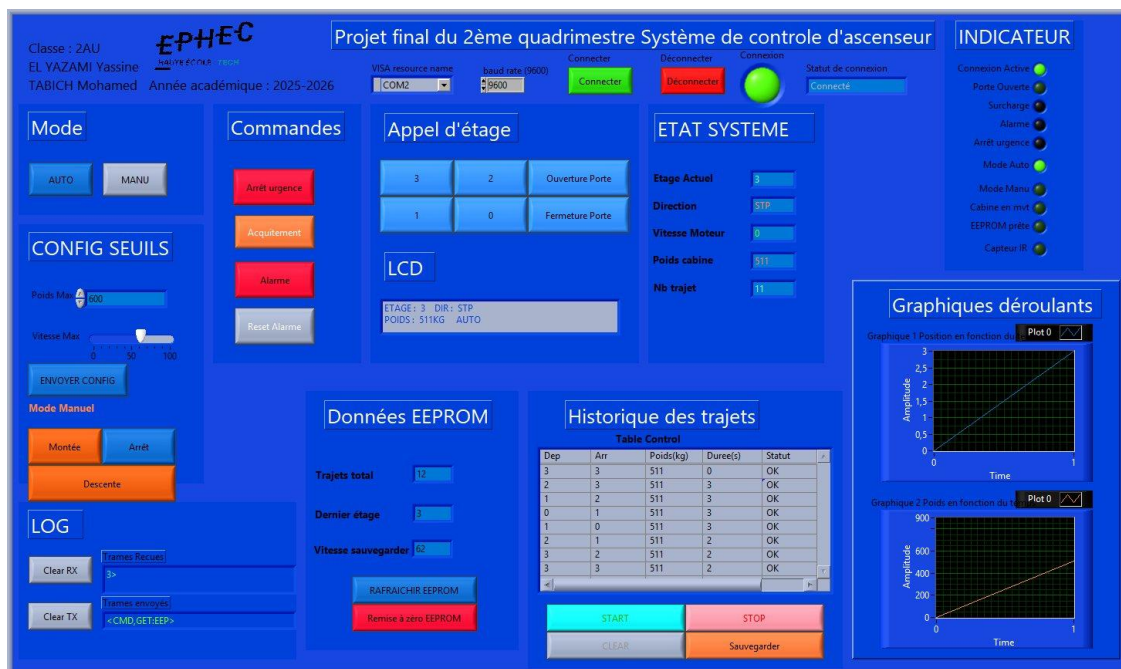


Figure 6 — Face-avant LabVIEW en fonctionnement (état connecté).

4.3 Diagramme de l'application avec analyse détaillée

Le diagramme est organisé autour de différents cas d'événements. Chaque événement vérifie d'abord que les conditions nécessaires sont remplies, comme l'établissement de la connexion, le mode de fonctionnement sélectionné ou l'absence d'une situation d'urgence. Si ces conditions sont respectées, la trame de commande correspondante est construite puis envoyée au PIC via la communication VISA. Les informations utiles sont également conservées dans les registres à décalage afin d'être réutilisées lors des itérations suivantes.

Les principaux événements et leur fonctionnement sont présentés dans les sections suivantes.

Événement Timeout — réception et traitement des trames

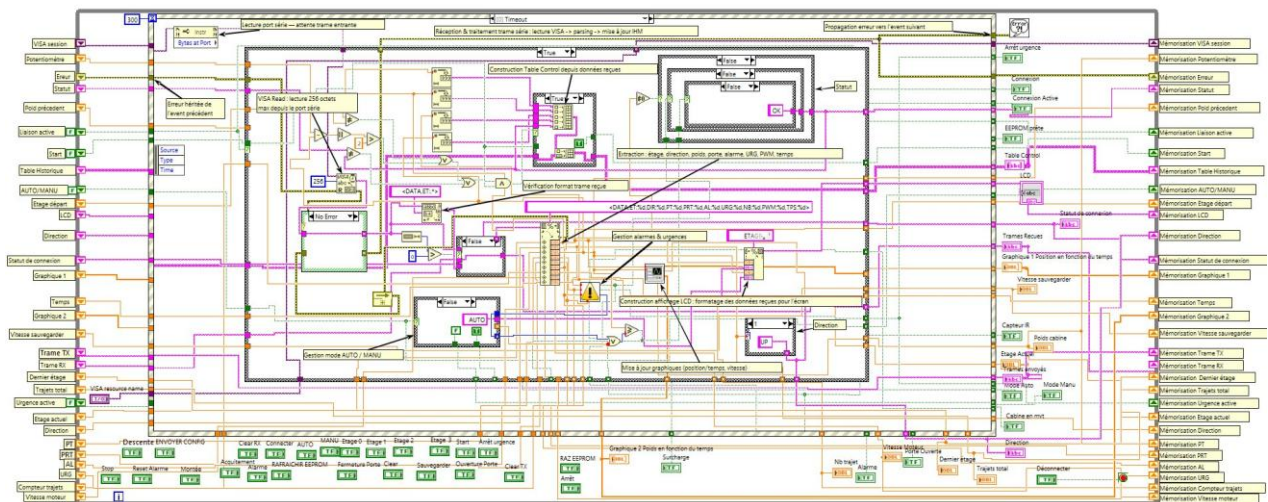


Figure 7 — Événement Timeout : lecture du port, analyse de la trame et mise à jour de l'IHM.

Cet événement, déclenché périodiquement, lit le port série (VISA Read), vérifie le format de la trame reçue, puis extrait les champs par « Scan From String » selon le motif de la trame DATA. Les valeurs (étage, direction, poids, porte, alarme, urgence, PWM, temps) alimentent les indicateurs, la simulation LCD, les graphiques déroulants et la construction de la ligne d'historique. La gestion des alarmes et urgences y est également centralisée. L'erreur VISA est propagée vers l'événement suivant.

Événements Connecter et Déconnecter

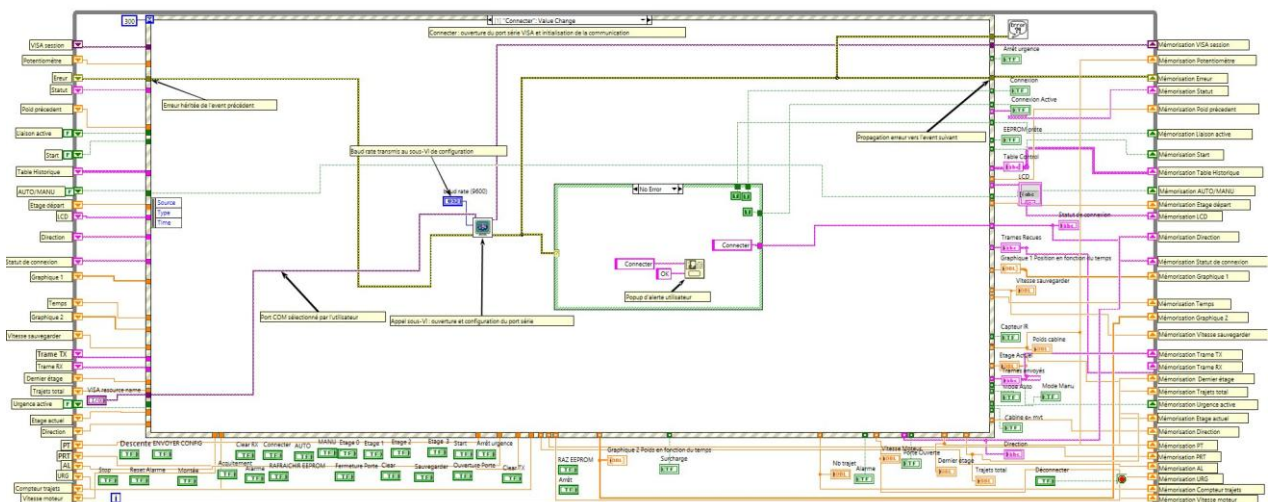


Figure 8 — Événement Connecter : ouverture et configuration du port série VISA.

Le cas « Connecter » appelle le sous-VI de communication qui ouvre et configure le port série (ressource VISA et débit sélectionnés par l'opérateur), met le statut à « Connecté » et active le voyant de connexion. Une boîte de dialogue confirme l'ouverture.

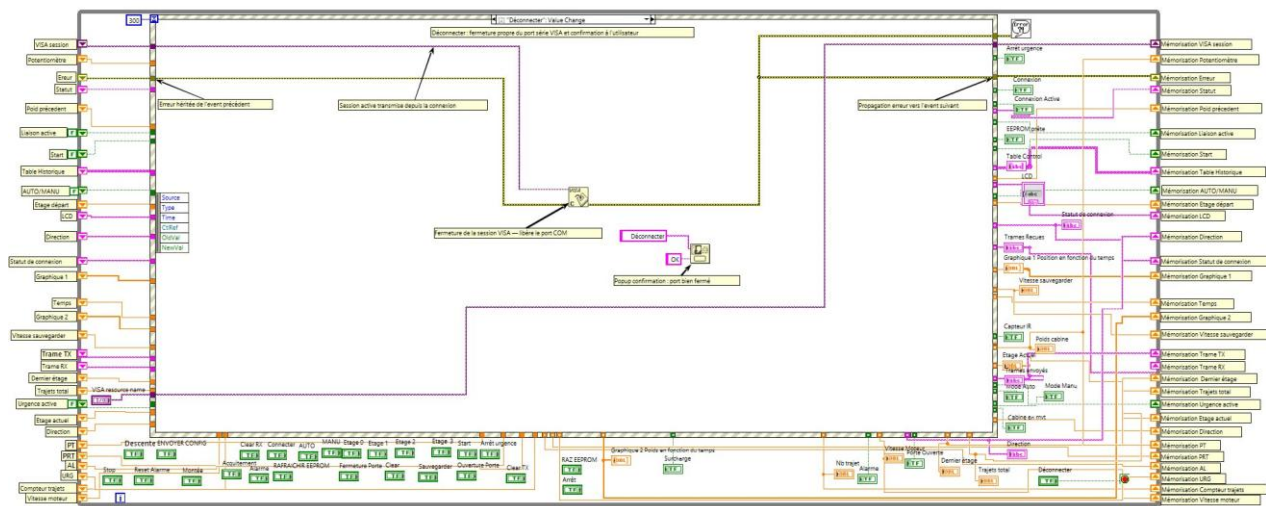


Figure 9 — Événement Déconnecter : fermeture propre de la session VISA.

Le cas « Déconnecter » ferme la session VISA pour libérer le port COM et confirme la fermeture à l'opérateur. La session active est transmise depuis l'événement de connexion via les registres à décalage.

Événements d'appel d'étage

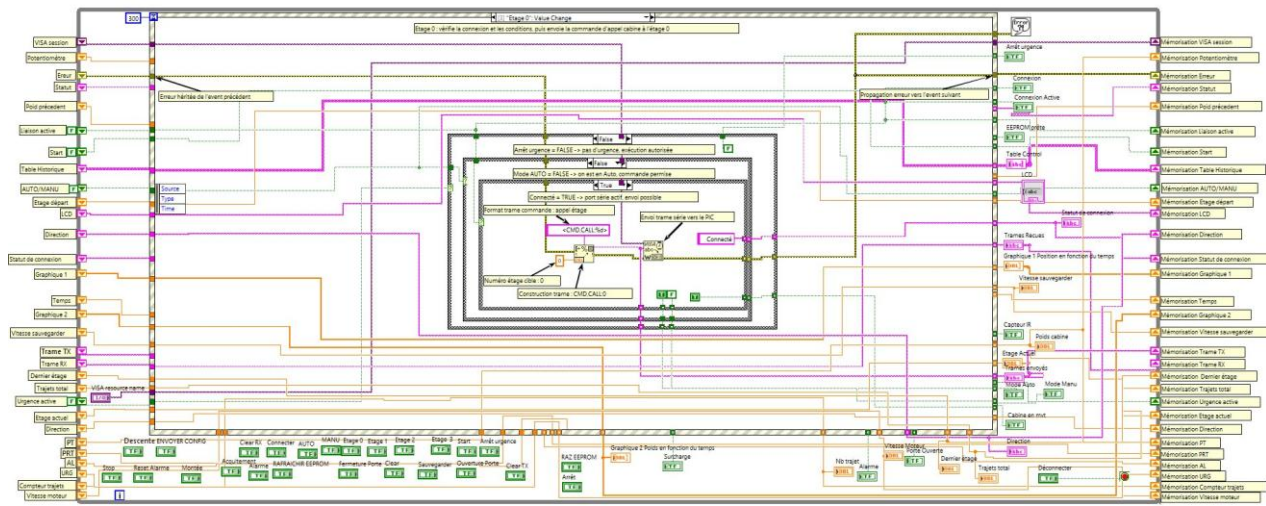


Figure 10 — Événement « Étage 0 » : envoi conditionnel de <CMD,CALL:0>.

Chaque bouton d'appel (0 à 3) déclenche un cas identique : si l'arrêt d'urgence est inactif, que le système est en mode automatique et que la liaison est connectée, l'application construit la trame <CMD,CALL:X> où X est le numéro d'étage, puis l'émet par VISA. Les cas des étages 1, 2 et 3 sont rigoureusement analogues, seul le numéro d'étage changeant.

Événements de mode AUTO / MANU

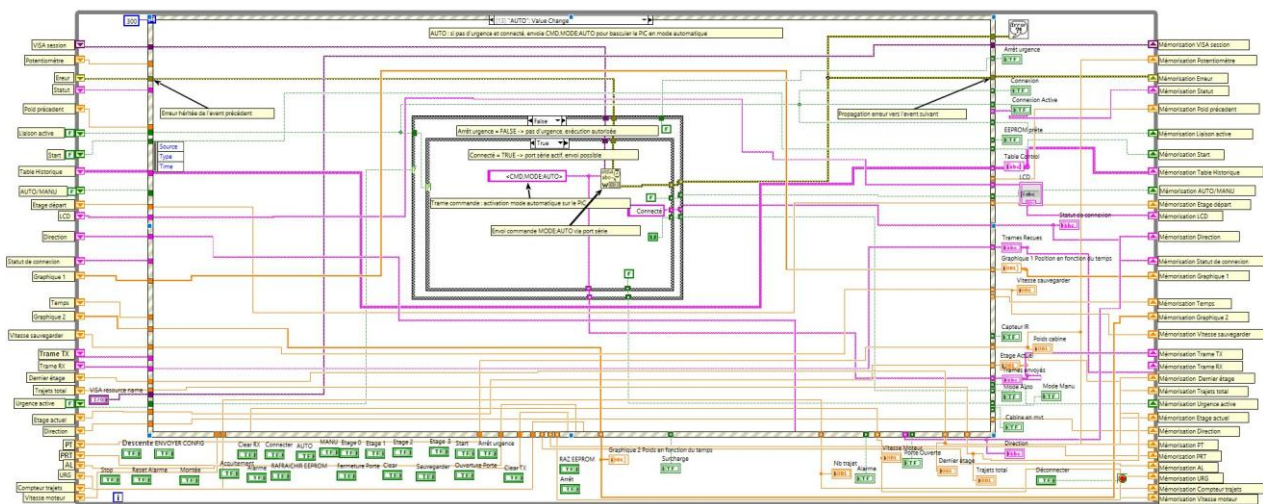


Figure 11 — Événement AUTO : envoi de <CMD,MODE: AUTO>.

Le cas « AUTO » envoie <CMD,MODE: AUTO> pour basculer le PIC en mode automatique, sous réserve d'absence d'urgence et de connexion active. Le cas « MANU » est symétrique et émet <CMD,MODE: MAN>. La sélection de mode conditionne l'autorisation des commandes manuelles décrites ci-après.

Configuration des seuils (Envoyer config)

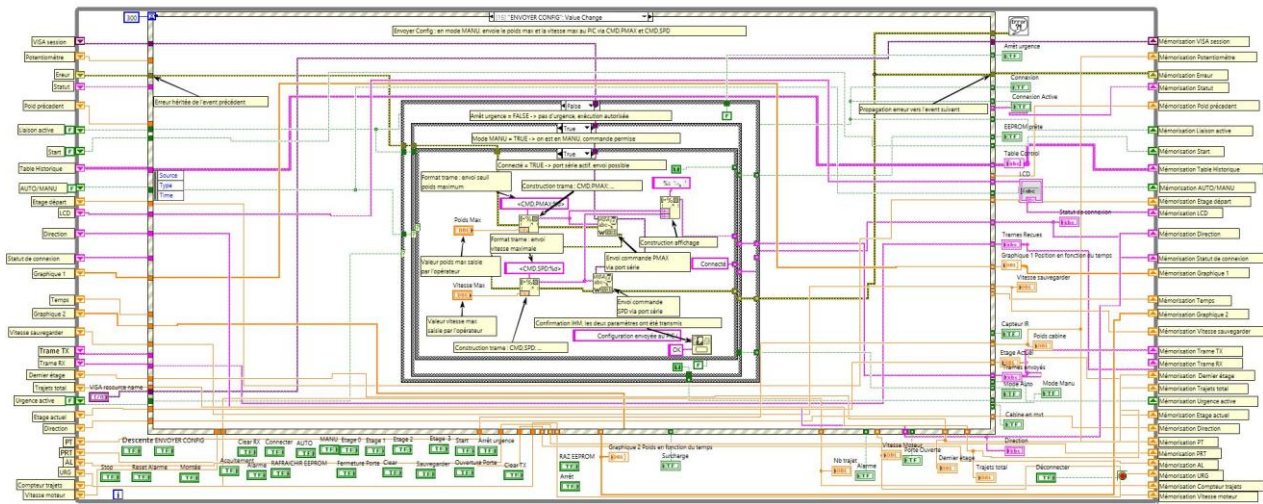


Figure 12 — Événement Envoyer config : émission de <CMD,PMAX:...> et <CMD,SPD:...>.

En mode manuel, ce cas transmet successivement le poids maximal saisi par l'opérateur (<CMD,PMAX:X>) et la vitesse maximale (<CMD,SPD:X>), puis affiche une confirmation que les deux paramètres ont été transmis au PIC.

Commandes manuelles du moteur et des portes

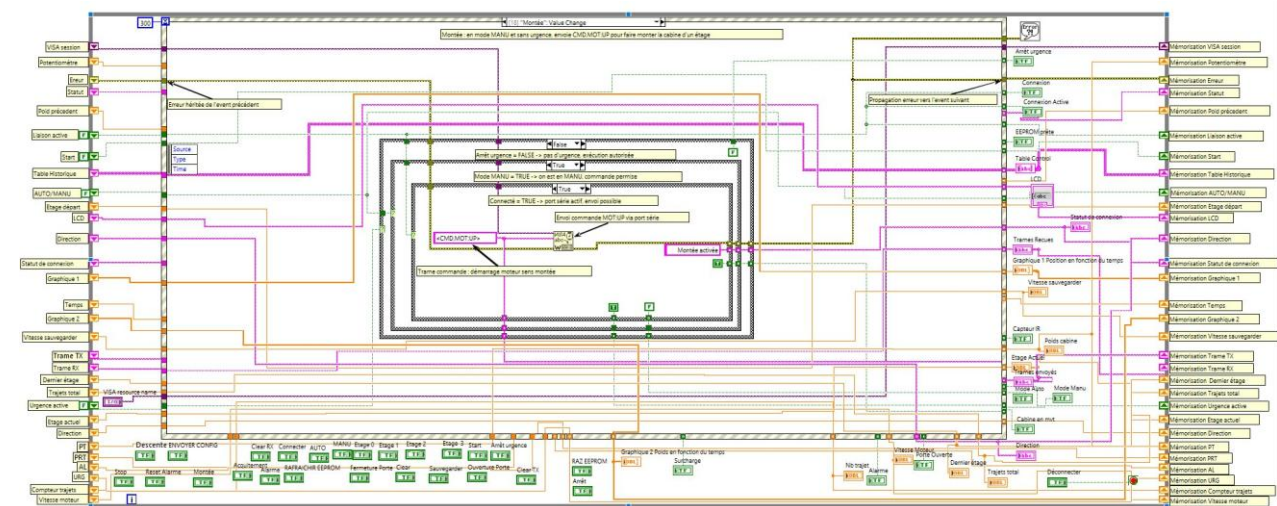


Figure 13 — Événement Montée : envoi de <CMD,MOT:UP>.

Le cas « Montée » émet <CMD,MOT:UP> pour faire monter la cabine d'un étage en mode manuel ; le cas « Descente » (figure ci-dessous) émet <CMD,MOT:DWN>.

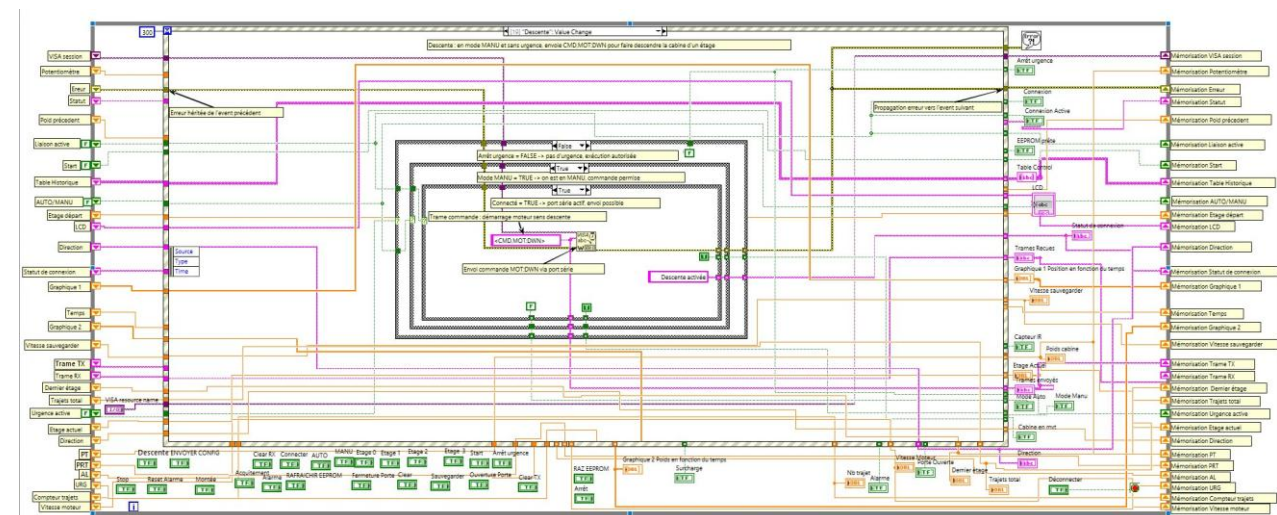


Figure 14 — Événement Descente : envoi de <CMD,MOT:DWN>.

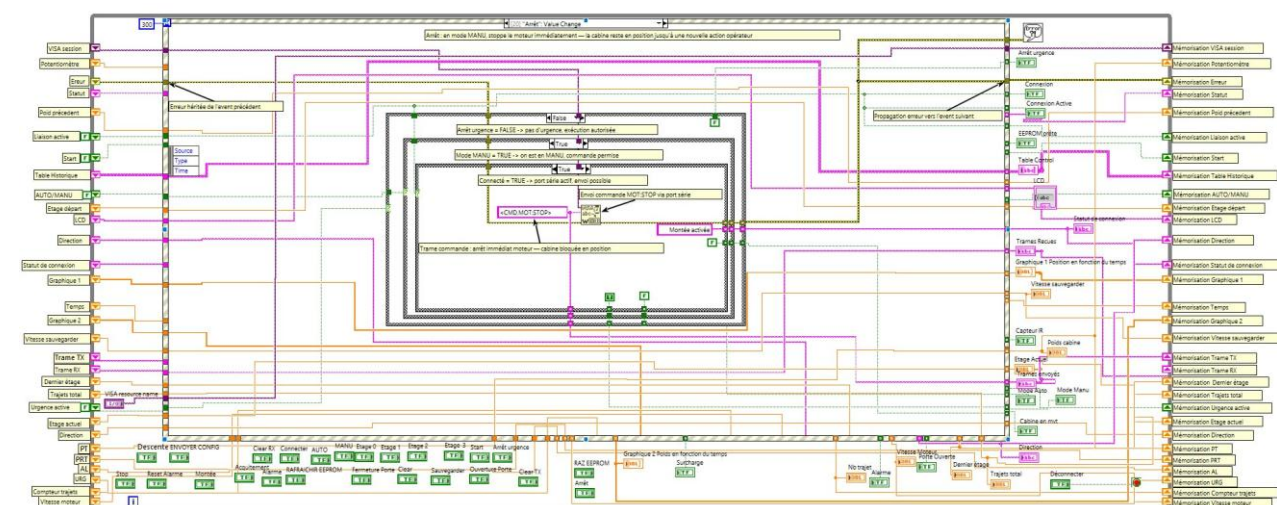


Figure 15 — Événement Arrêt : envoi de <CMD,MOT:STOP> (arrêt propre du moteur).

Le cas « Arrêt » stoppe proprement le moteur (<CMD,MOT:STOP>) ; la cabine reste alors en position jusqu'à une nouvelle action de l'opérateur. Les cas « Ouverture porte » et « Fermeture porte » émettent respectivement <CMD.DOOR:O> et <CMD.DOOR:F>.

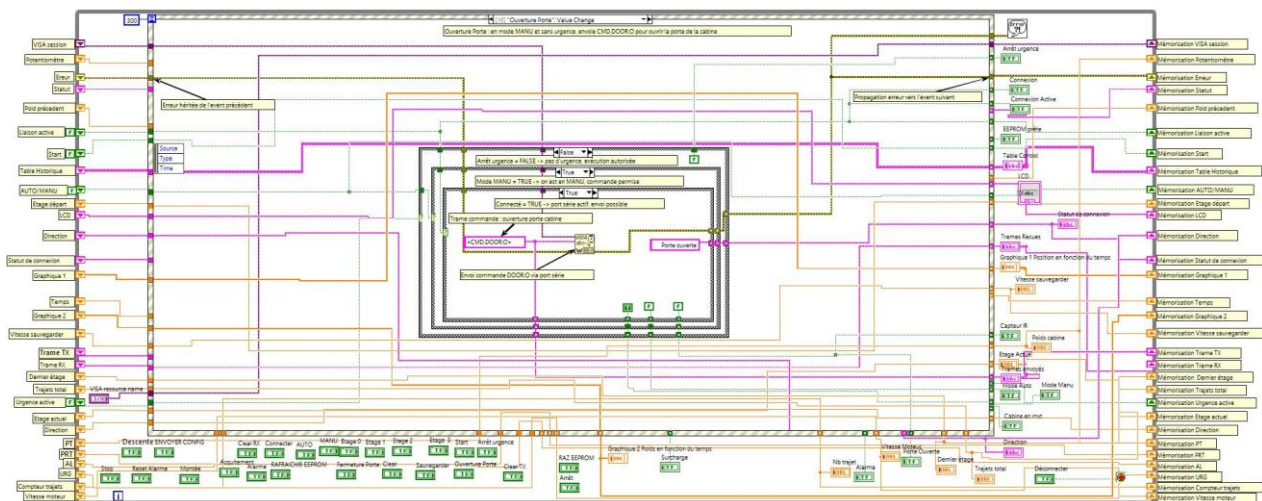


Figure 16 — Événement Ouverture porte : envoi de <CMD,DOOR:O>.

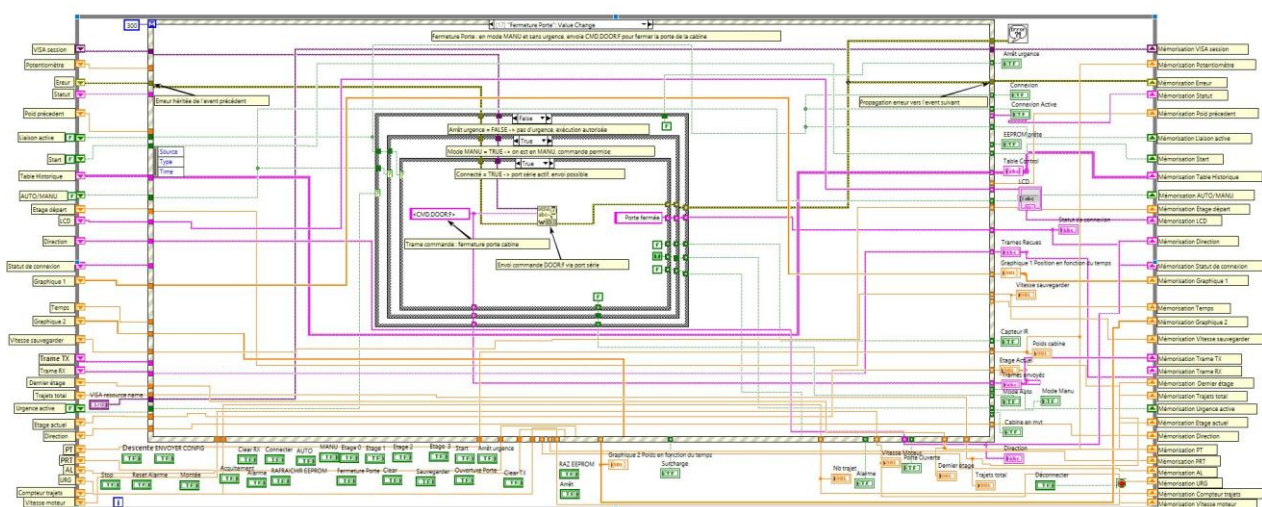


Figure 17 — Événement Fermeture porte : envoi de <CMD,DOOR:F>.

Sécurité : urgence, acquittement, alarme et reset

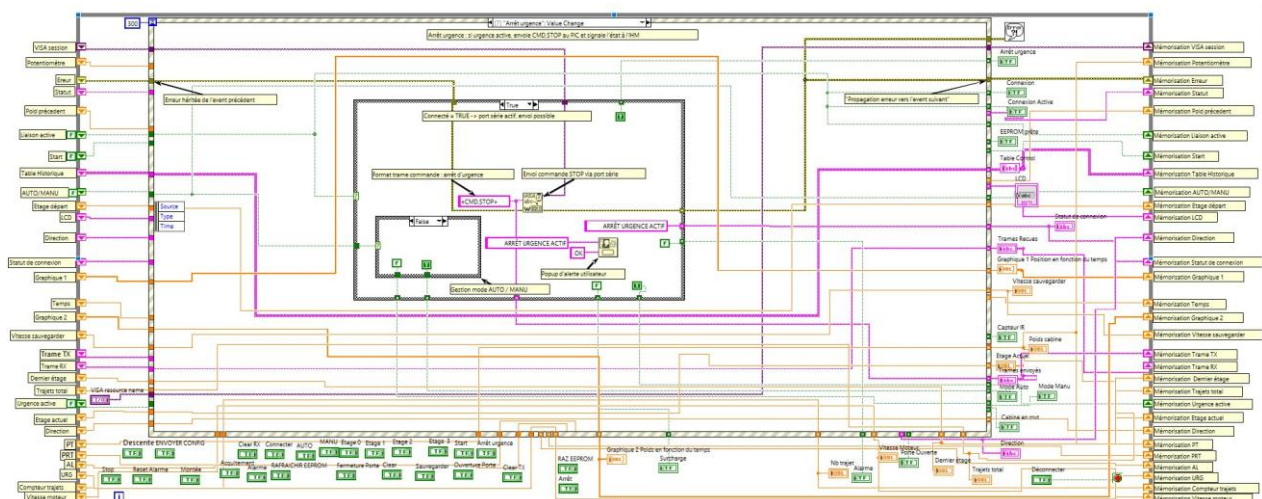


Figure 18 — Événement Arrêt urgence : envoi de <CMD,STOP>.

Le cas « Arrêt urgence » émet <CMD,STOP> et signale l'état d'urgence sur l'IHM.

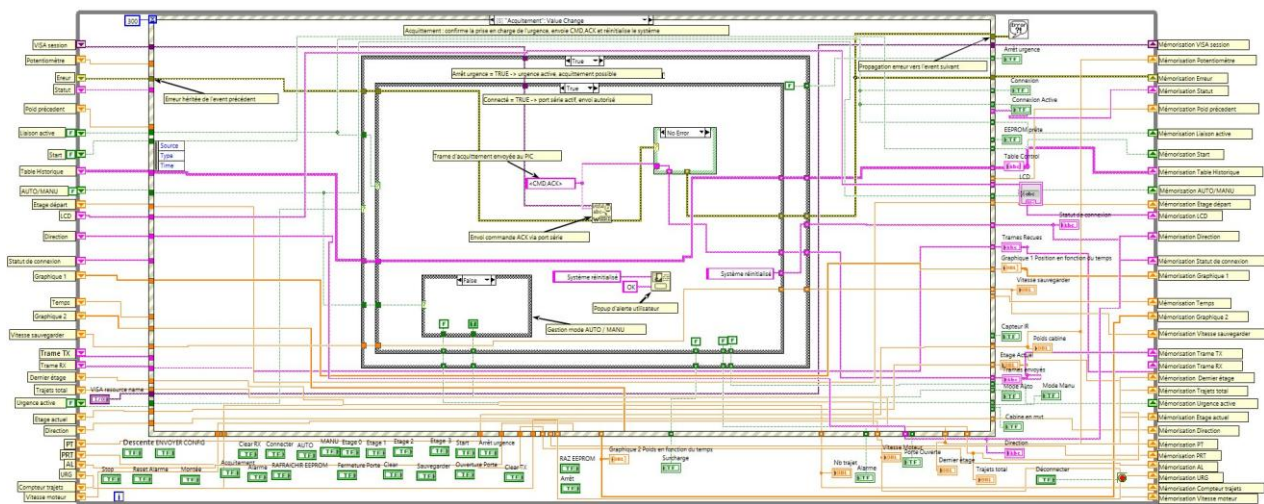


Figure 19 — Événement Acquittement : envoi de <CMD,ACK>.

Le cas « Acquittement » confirme la prise en charge de l'urgence (<CMD,ACK>) et réinitialise l'affichage du système.

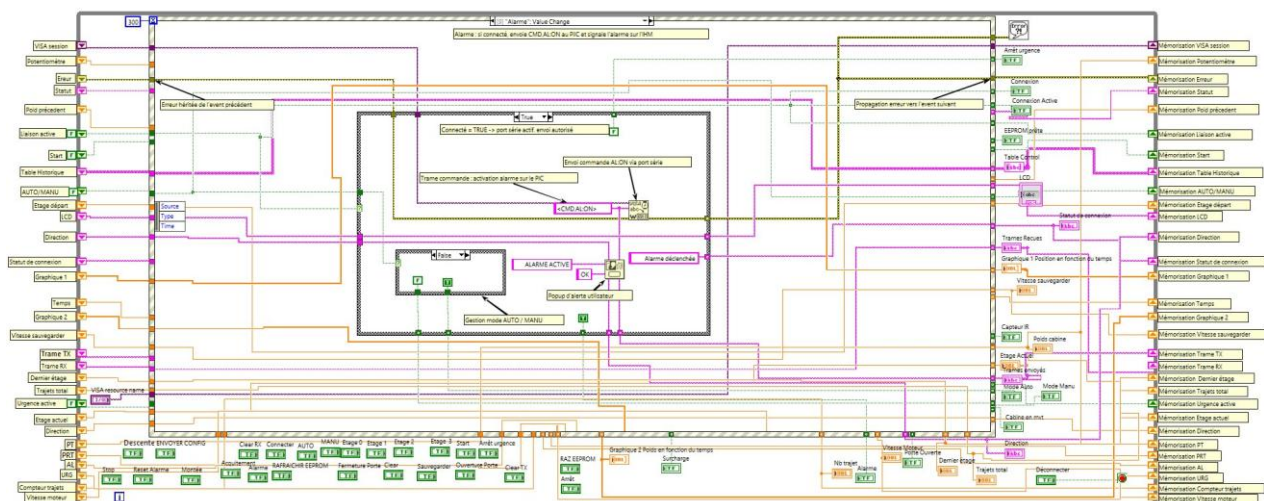


Figure 20 — Événement Alarme : envoi de <CMD,AL:ON>.

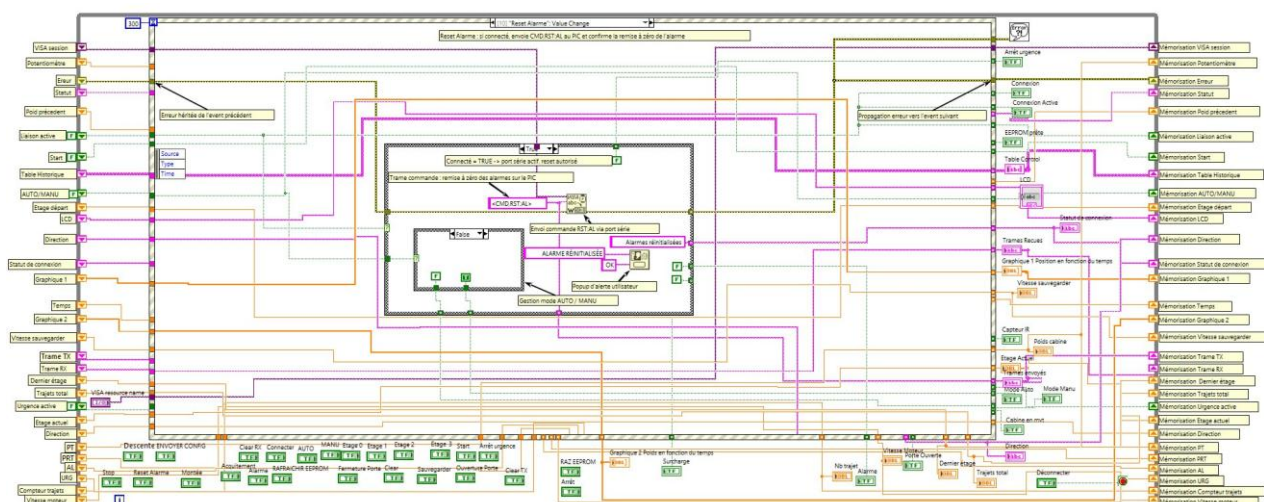


Figure 21 — Événement Reset Alarme : envoi de <CMD,RST:AL>.

Les cas « Alarme » et « Reset Alarme » émettent respectivement <CMD,AL:ON> et <CMD,RST:AL>, en signalant le changement d'état correspondant sur l'IHM.

Gestion de la mémoire EEPROM

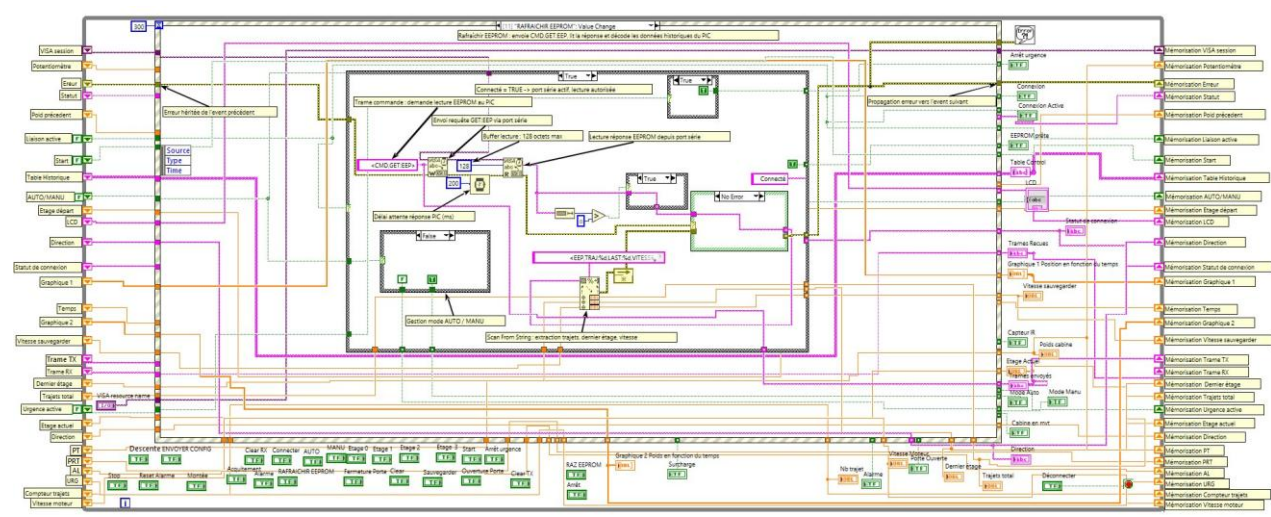


Figure 22 — Événement Rafraîchir EEPROM : requête <CMD,GET:EEP> et décodage de la réponse.

Le cas « Rafraîchir EEPROM » envoie <CMD,GET:EEP>, attend la réponse du PIC, puis décode la trame <EEP,TRAJ:...,LAST:...,VITESSE:...> par « Scan From String » pour mettre à jour les indicateurs de trajets totaux, dernier étage et vitesse sauvegardée.

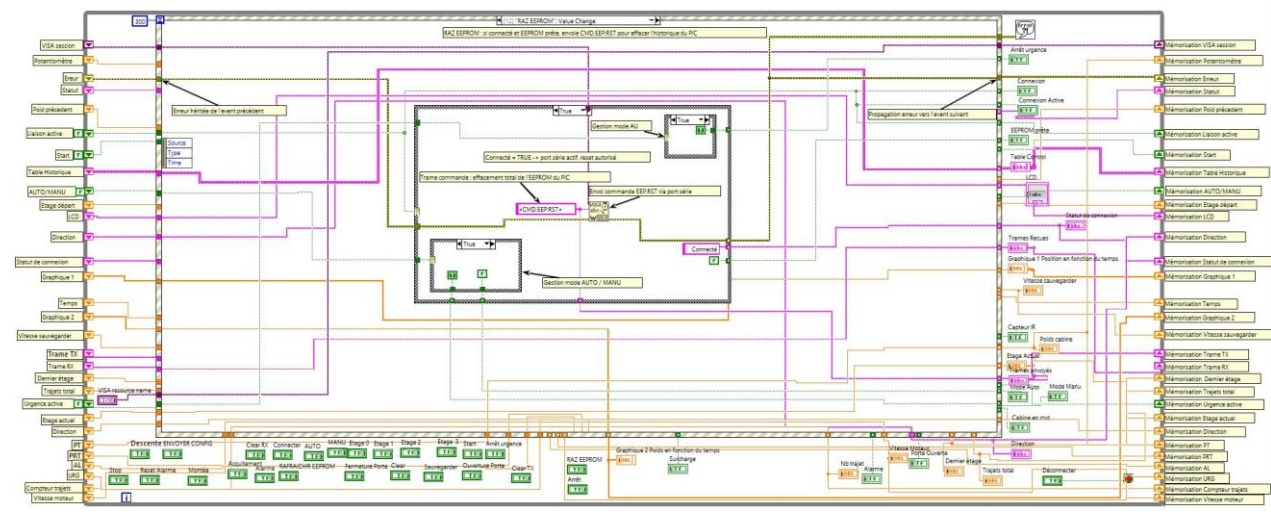


Figure 23 — Événement RAZ EEPROM : envoi de <CMD,EEP:RST>.

Le cas « RAZ EEPROM » émet <CMD,EEP:RST> pour effacer l'historique mémorisé dans le PIC, sous réserve de connexion et d'EEPROM prête.

Gestion de l'acquisition et de l'historique

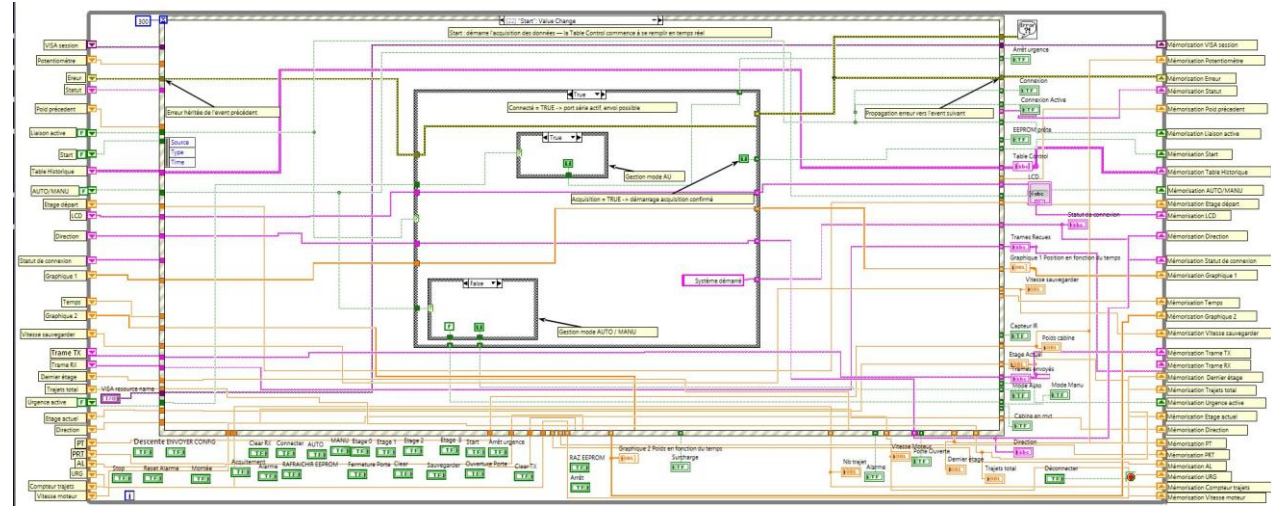


Figure 24 — Événement Start : démarrage de l'acquisition (remplissage de la Table Control).

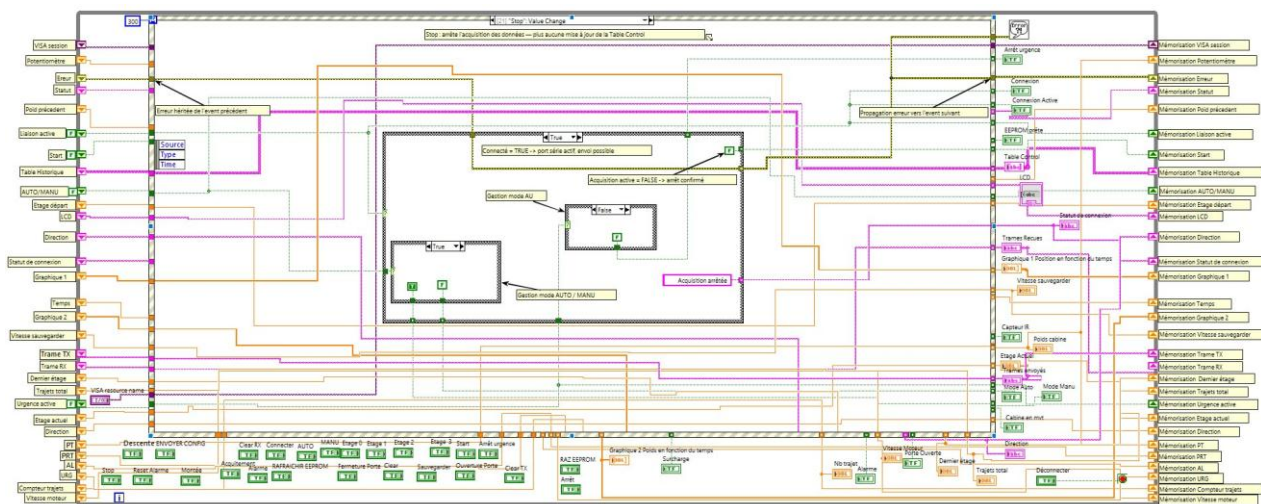


Figure 25 — Événement Stop : arrêt de l'acquisition.

Les cas « Start » et « Stop » activent ou désactivent le remplissage en temps réel du tableau d'historique. Le cas « Clear » vide entièrement ce tableau et remet les compteurs à zéro.

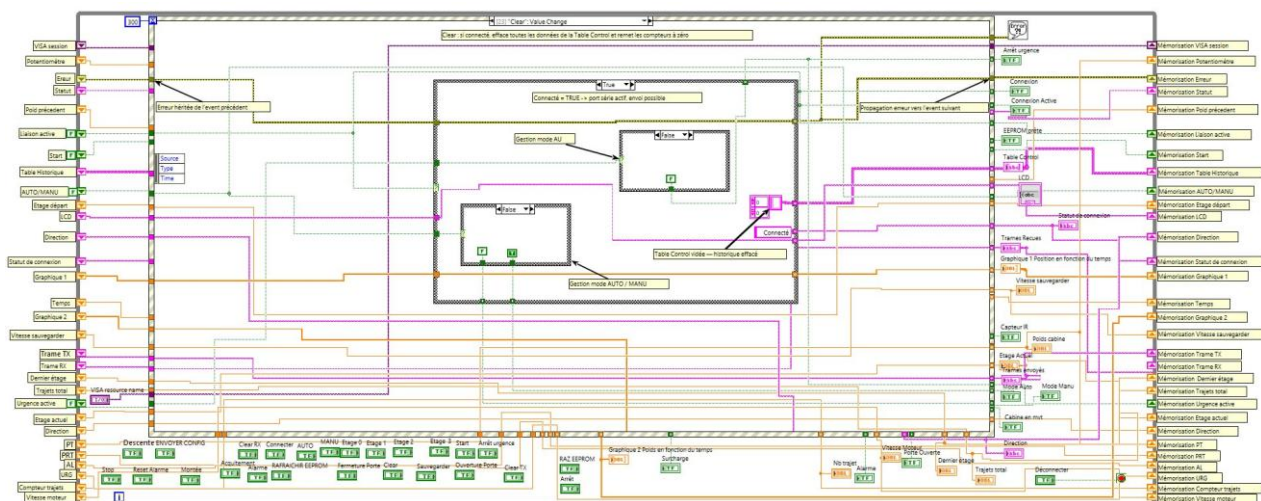


Figure 26 — Événement Clear : effacement du tableau d'historique.

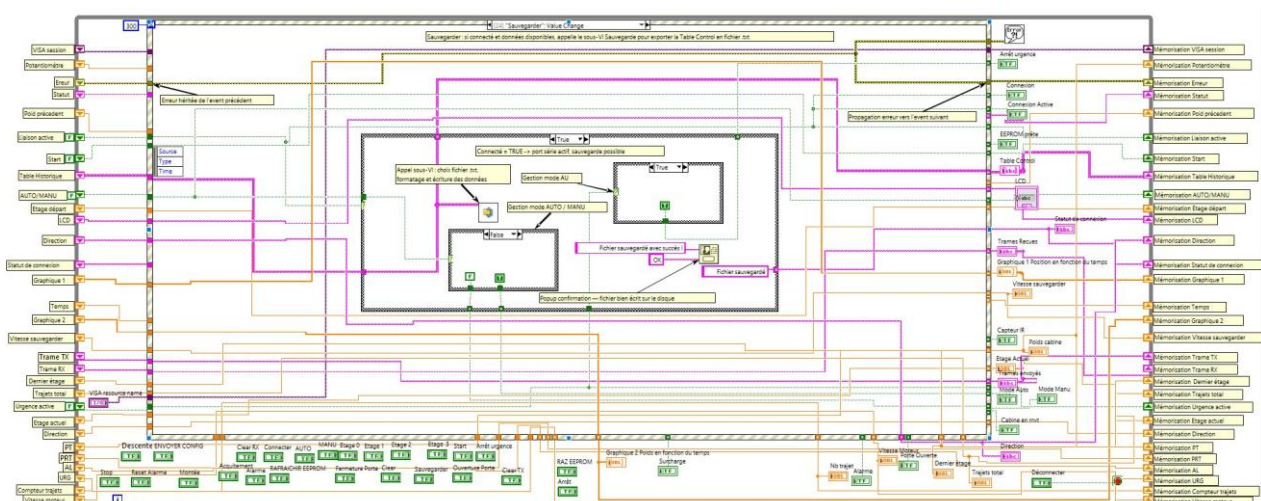


Figure 27 — Événement Sauvegarder : appel du sous-VI d'export vers fichier.

Le cas « Sauvegarder » appelle le sous-VI de sauvegarde qui exporte le contenu du tableau d'historique vers un fichier texte, puis confirme l'écriture. Enfin, les cas « Clear RX » et « Clear TX » effacent respectivement les indicateurs de trames reçues et envoyées du journal.

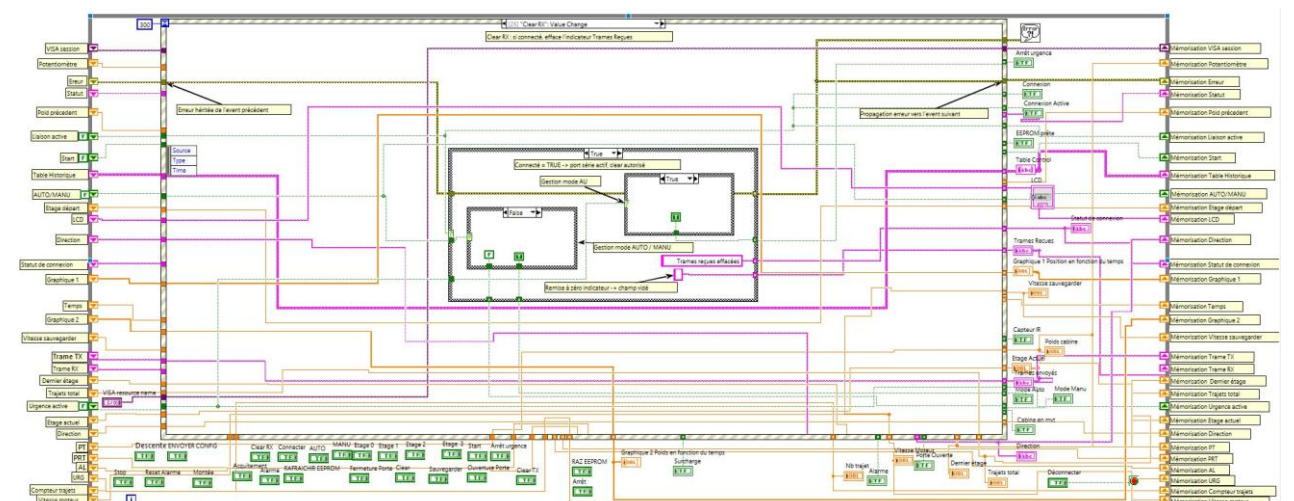


Figure 28 — Événement Clear RX : effacement de l'indicateur des trames reçues.

4.4 Intégration avec le microcontrôleur

L'intégration repose sur un échange permanent entre le PIC et l'interface LabVIEW. Dans le sens PIC vers PC, l'événement **Timeout** lit régulièrement le port série et décode les trames **DATA** et **EEP** envoyées par le microcontrôleur. Les informations reçues sont ensuite utilisées pour mettre à jour les indicateurs, les graphiques et les tableaux de la face avant.

Dans le sens PC vers PIC, chaque action de l'opérateur provoque l'envoi d'une trame **CMD**, dont l'exécution est confirmée par un acquittement du microcontrôleur.

La correspondance entre les données reçues et les éléments affichés est directe : **ET** met à jour l'étage actuel, **DIR** la direction, **PT** le poids de la cabine, **PRT** l'état de la porte, et ainsi de suite. Grâce à cet échange continu, l'interface LabVIEW affiche en permanence l'état réel du système embarqué.

4.5 Traitement des données acquises

Le traitement des données commence par l'analyse des trames reçues grâce à la fonction **Scan From String**, qui extrait automatiquement les différentes valeurs contenues dans le message.

Les informations de position et de poids sont ensuite utilisées pour alimenter deux graphiques déroulants, permettant de suivre leur évolution au fil du temps. Lorsqu'un trajet est terminé, une nouvelle ligne est ajoutée au tableau d'historique avec les principales informations du déplacement : étage de départ, étage d'arrivée, poids, durée du trajet et statut.

Enfin, un sous-VI dédié permet d'enregistrer cet historique dans un fichier texte. Cette fonctionnalité répond à l'exigence de sauvegarde des données dans un fichier texte ou CSV et facilite leur consultation ou leur analyse par la suite.

4.6 Conclusion partielle (A208)

L'application LabVIEW remplit pleinement son rôle de poste de supervision : elle gère proprement le port série via VISA, présente une face-avant claire et complète, traite les données reçues pour les visualiser et les archiver, et offre un jeu complet de commandes acquittées. Son architecture événementielle et modulaire, structurée en sous-VI, en fait une application lisible et maintenable, fidèle aux bonnes pratiques attendues.

5. Tests et résultats

Le système a été validé en deux étapes : d'abord en simulation sous Proteus, puis sur la carte physique EasyPIC V7, en liaison avec l'interface LabVIEW. La simulation a permis de vérifier la logique de commande, le décodage du protocole et l'enchaînement des séquences de déplacement sans risque matériel ; la carte réelle a confirmé le fonctionnement effectif, exigence essentielle du projet.

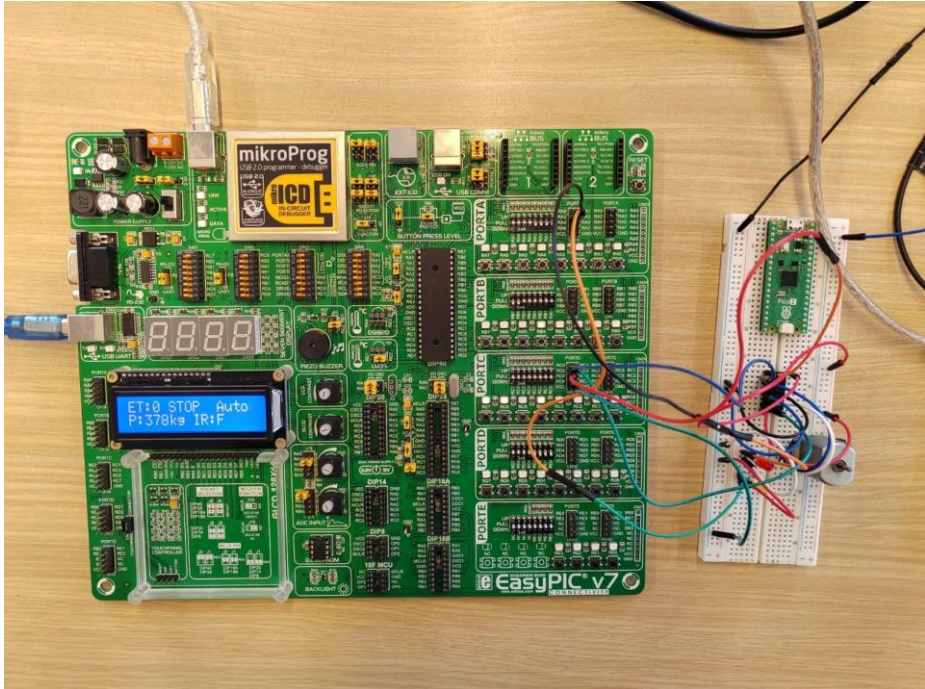


Figure 29 — Montage réel sur carte EasyPIC V7 (afficheur indiquant « ET:0 STOP Auto / P:378kg IR:F »).

La photographie ci-dessus montre la carte en fonctionnement. L'écran LCD affiche l'étage actuel, l'état du système, le mode de fonctionnement, le poids mesuré ainsi que l'état de la porte, ce qui permet de vérifier que le programme s'exécute correctement. Le moteur de translation, installé sur une platine d'essai et commandé par le pont en H, reproduit les déplacements demandés par le système.

La figure 6, qui présente l'interface LabVIEW connectée au système, montre l'ensemble de la chaîne en fonctionnement. Les informations envoyées par le PIC sont correctement reçues et affichées : position de la cabine, évolution du poids, historique des trajets et état général du système. Les données enregistrées dans l'EEPROM, comme le nombre total de trajets, le dernier étage desservi et la vitesse sauvegardée, sont également mises à jour correctement.

Le tableau suivant reprend les principaux scénarios de test réalisés ainsi que les résultats obtenus.

Scénario testé	Résultat attendu	Résultat obtenu
Appel d'étage en mode automatique	Déplacement vers l'étage, ouverture de porte, trajet enregistré	Conforme
Montée/descente en mode manuel	Mouvement d'un étage sur commande, arrêt propre	Conforme
Arrêt d'urgence (bouton et PC)	Coupure immédiate du moteur, attente d'acquiescement	Conforme
Alarme et reset	Blocage, signalisation, reprise après acquiescement	Conforme
Surcharge (poids $\geq$ seuil)	Refus de déplacement, message et voyant de surcharge	Conforme

Scénario testé	Résultat attendu	Résultat obtenu
Porte ouverte au démarrage	Refus de départ, message « Porte ouverte »	Conforme
Configuration PMAX / SPD	Prise en compte des seuils, vitesse mémorisée en EEPROM	Conforme
GET:EEP / EEP:RST	Lecture puis remise à zéro des données EEPROM	Conforme

6. Pistes d'amélioration

Plusieurs améliorations pourraient être apportées au système pour le rapprocher davantage d'une installation réelle et améliorer sa fiabilité :

- Remplacer l'estimation de la position par temporisation par des capteurs de fin de course ou un encodeur. La position de la cabine serait alors connue avec précision, même après un arrêt en cours de déplacement.
- Améliorer la gestion des appels d'étage en utilisant un algorithme plus évolué, capable d'optimiser l'ordre de desserte des différents étages.
- Ajouter un mécanisme de vérification des trames, comme un checksum ou un CRC, afin de détecter d'éventuelles erreurs de transmission en plus des acquittements déjà présents.
- Associer une date et une heure à chaque trajet et permettre l'export de l'historique dans un fichier CSV structuré, voire dans une base de données, afin de faciliter l'analyse des données.
- Enrichir l'interface LabVIEW avec de nouveaux graphiques, un journal détaillé des événements ou encore des alarmes sonores. Il serait également possible d'étendre la supervision à plusieurs cabines.
- Renforcer la gestion des changements de mode et mettre en place une surveillance de la communication afin de détecter rapidement une perte de liaison entre le PC et le microcontrôleur.

7. Conclusion générale

Ce projet a permis d'intégrer, autour d'une application concrète, les compétences développées dans les deux unités d'enseignement concernées. La partie embarquée met en œuvre l'essentiel des périphériques du PIC18F45K22 — interruptions multiples, ADC, PWM, I²C, UART et LCD — au sein d'une architecture réactive et sûre, capable de gérer les modes automatique et manuel ainsi que les situations dégradées. La partie acquisition fournit une interface de supervision LabVIEW modulaire, qui dialogue avec le microcontrôleur selon un protocole bidirectionnel acquitté, visualise les données en temps réel et les archive.

Au-delà des aspects purement techniques, le projet a constitué un exercice de conception système complet : définition d'un protocole de communication, répartition des responsabilités entre le PIC et le PC, gestion des cas limites et validation progressive en simulation puis sur matériel. Le système final est fonctionnel sur la carte EasyPIC V7, conformément à l'objectif central de l'énoncé. Les pistes d'amélioration identifiées ouvrent par ailleurs des perspectives intéressantes pour une éventuelle industrialisation.

Démonstration du système de contrôle automatique d'un ascenseur — EL YAZAMI Yassine & TABICH Mohamed — EPHEC 2025-2026

8. Références bibliographiques

- [1] Microchip Technology, PIC18F45K22 — Datasheet, document DS40001412.
- [2] MikroElektronika, EasyPIC v7 Development Board — User Manual.
- [3] MikroElektronika, mikroC PRO for PIC — Compiler Reference & Libraries.
- [4] Texas Instruments, L293D Quadruple Half-H Driver — Datasheet.
- [5] Fairchild / ON Semiconductor, NM24C08 — 8K-bit Serial EEPROM (I²C) — Datasheet.
- [6] National Instruments, LabVIEW & NI-VISA — Serial Communication User Guide.
- [7] E. COSTA, F. DONNAY, Projet final du 2^e quadrimestre — A208 / A211, EPHEC Tech, 2025-2026.

9. Annexes — Code source complet

Le code source complet du firmware MikroC est reproduit ci-dessous. La version intégralement commentée ainsi que l'ensemble des fichiers du projet (MikroC, schéma Proteus et sources LabVIEW) sont disponibles sur le dépôt GitHub du binôme.

```

1  #define SIMULATION_PROTEUS
2
3  // Connexions de l'afficheur LCD
4  sbit LCD_RS at RB4_bit;
5  sbit LCD_EN at RB5_bit;
6  sbit LCD_D4 at RB0_bit;
7  sbit LCD_D5 at RB1_bit;
8  sbit LCD_D6 at RB2_bit;
9  sbit LCD_D7 at RB3_bit;
10 sbit LCD_RS_Direction at TRISB4_bit;
11 sbit LCD_EN_Direction at TRISB5_bit;
12 sbit LCD_D4_Direction at TRISB0_bit;
13 sbit LCD_D5_Direction at TRISB1_bit;
14 sbit LCD_D6_Direction at TRISB2_bit;
15 sbit LCD_D7_Direction at TRISB3_bit;
16
17 // LEDs d'etat
18 #define LED1      LATA2_bit
19 #define LED2      LATA3_bit
20
21 // Boutons poussoirs
22 #define BP_URGENCE PORTB.F6
23 #define BP_ALARME  PORTB.F7
24 #define BP_ACQ     PORTD.F4
25
26 // Actions moteur et etat actif
27 #define MOTEUR_MONTER()    do { LATC0_bit = 1; LATC1_bit = 0; moteur_actif = 1; }
28 while(0)
29 #define MOTEUR_DESCENDRE() do { LATC0_bit = 0; LATC1_bit = 1; moteur_actif = 1;
30 } while(0)
31 #define MOTEUR_ARRETER()   do { LATC0_bit = 0; LATC1_bit = 0; PWM1_Set_Duty(0);
32 moteur_actif = 0; pwm_actuel = 0; } while(0)
33
34 // Parametres pour les rampes PWM
35 #define PWM_MIN      80
36 #define PWM_PALIERS  6
37
38 // Temporisations adaptees pour Proteus ou le reel
39 #ifdef SIMULATION_PROTEUS
40     #define MS_PALIER          20
41     #define MS_CROISIERE      500
42     #define MS_FERMETURE       50
43     #define MS_OUVERTURE      100
44     #define MS_ARRET_INTERMED  500
45     #define MS_PWM_PAUSE       0
46     #define MS_LOOP_STEP      20
47 #else
48     #define MS_PALIER          150
49     #define MS_CROISIERE      1800
50     #define MS_FERMETURE       300
51     #define MS_OUVERTURE      600
52     #define MS_ARRET_INTERMED 3000
53     #define MS_PWM_PAUSE       5

```

```

54     #define MS_LOOP_STEP          50
55 #endif
56
57 // Ajustement temps de croisiere pour 1 seul etage
58 #define MS_CROISIERE_1ET  (MS_CROISIERE - PWM_PALIER * MS_PALIER)
59
60 #define _FOSC_HZ          8000000UL
61
62 // Configuration de la base de temps du Timer0 (1 seconde)
63 #define T0_TICKS_1S      (_FOSC_HZ / 1024UL)
64 #define T0_RELOAD        (65536UL - T0_TICKS_1S)
65 #define T0_RELOAD_H      ((unsigned char)((T0_RELOAD >> 8) & 0xFF))
66 #define T0_RELOAD_L      ((unsigned char)(T0_RELOAD & 0xFF))
67
68 // Constantes de l'application
69 #define NB_ETAGES        4
70 #define POIDS_MAX_DEF    630
71 #define POT_ADC_MAX      900
72
73 // Adresses de l'EEPROM I2C
74 #define EEPROM_W         0xA0
75 #define EEPROM_R         0xA1
76
77 // Variables globales de suivi de la cabine
78 unsigned char etage_actuel      = 0;
79 unsigned char etage_cible      = 0;
80 unsigned char en_mouvement     = 0;
81 char         direction         = 'S';
82 unsigned int  poids_kg         = 0;
83 unsigned char ir_porte         = 0;
84 unsigned char req[NB_ETAGES]   = {0,0,0,0};
85
86 // Variables pour la securite et les modes
87 unsigned char al_active        = 0;
88 unsigned char urg_active      = 0;
89 unsigned char mode_auto       = 1;
90
91 // Variables pour le contrôle du poids
92 unsigned int  poids_max       = POIDS_MAX_DEF;
93 unsigned int  seuil_surge     = POIDS_MAX_DEF;
94 unsigned char surcharge_active = 0;
95
96 // Compteurs de suivi
97 unsigned int  nb_session      = 0;
98 unsigned int  nb_trajets      = 0;
99
100 // Sauvegardes et limites de vitesse
101 unsigned char last_etage      = 0;
102 unsigned char vitesse_max_pc  = 100;
103 unsigned char vitesse_eeeprom = 100;
104 unsigned char pwm_max_eff     = 255;
105
106 // Variables de mesure temps/vitesse
107 unsigned char pwm_actuel      = 0;
108 unsigned int  temps_trajet    = 0;
109
110 // Flags de positionnement et verrouillage
111 unsigned char position_inconnue = 0;
112 unsigned char entre_etages      = 0;

```

```

113 unsigned char porte_cmd          = 0;
114 unsigned char moteur_actif       = 0;
115
116 unsigned char etat_surcharge     = 0;
117
118 // Flags de communication avec les interruptions
119 volatile unsigned char timer0_flag = 0;
120 volatile unsigned int  timer0_count = 0;
121 volatile unsigned char urgence_flag = 0;
122 volatile unsigned char al_flag      = 0;
123
124 volatile unsigned char suppress_data_count = 0;
125
126 // Parametres de la file d'attente UART
127 #define CMD_QSIZE 4
128 #define CMD_QLEN 40
129
130 volatile char          rx_buf[50];
131 volatile unsigned char rx_idx  = 0;
132
133 volatile char          cmd_queue[CMD_QSIZE][CMD_QLEN];
134 volatile unsigned char cmd_qhead = 0;
135 volatile unsigned char cmd_qtail = 0;
136
137 volatile unsigned char stop_demande = 0;
138
139 // Buffers memoire pour les deux lignes du LCD
140 char l1[20];
141 char l2[20];
142
143 // Déclarations des fonctions
144 void recalc_pwm_max();
145 unsigned char pop_cmd(char *dest);
146 void eeeprom_charger();
147 void eeeprom_sauver_trajet();
148 void eeeprom_reset();
149 void set_pwm(unsigned char duty);
150
151 void uart_send_data();
152 void uart_send_eeeprom();
153 void uart_ack_ok();
154 void uart_ack_err();
155
156 void lire_capteurs();
157 void gerer_leds();
158 void maj_surcharge(unsigned char force_transition);
159 char etat_porte_char();
160
161 void lcd_build_surcharge_l2(char *buf);
162 void afficher_lcd();
163 void lcd_transition();
164 void lcd_update_transit();
165
166 void appliquer_pmax(unsigned int val);
167 void appliquer_spd(unsigned char val);
168
169 void attendre_ms(unsigned int ms);
170
171 void rampe_accel();

```

```

172 void rampe_decel();
173 void demarrer_moteur(char sens);
174
175 void scanner_req();
176 void vider_req();
177 unsigned char prochain_req();
178
179 void deplacer_vers(unsigned char cible);
180 void parser_cmd(char *buf);
181
182 // Routine d'interruption
183 void interrupt() {
184     char c;
185
186     // Gestion du rafraîchissement par le Timer0
187     if (TMR0IE_bit && TMR0IF_bit) {
188         TMR0IF_bit = 0;
189         TMR0H = T0_RELOAD_H;
190         TMR0L = T0_RELOAD_L;
191         timer0_flag = 1;
192         timer0_count++;
193     }
194
195     // Gestion des evenements sur les boutons poussoirs du PORTB
196     if (RBIE_bit && RBIF_bit) {
197         unsigned char pb = PORTB;
198         RBIF_bit = 0;
199
200         // Detection Urgence
201         if ((pb & 0x40) && !urg_active) {
202             LATC0_bit = 0;
203             LATC1_bit = 0;
204             moteur_actif = 0;
205             pwm_actuel = 0;
206             urg_active = 1;
207             en_mouvement = 0;
208             LED2 = 1;
209             urgence_flag = 1;
210         }
211
212         // Detection Alarme
213         if ((pb & 0x80) && !al_active) {
214             LATC0_bit = 0;
215             LATC1_bit = 0;
216             moteur_actif = 0;
217             pwm_actuel = 0;
218             al_active = 1;
219             al_flag = 1;
220             en_mouvement = 0;
221             LED2 = 1;
222         }
223     }
224
225     // Reception UART et stockage dans le buffer circulaire
226     if (RC1IE_bit && RC1IF_bit) {
227         unsigned char j;
228         unsigned char n;
229         unsigned char nxt;
230

```

```

231     if (OERR1_bit) {
232         CREN1_bit = 0;
233         CREN1_bit = 1;
234     }
235
236     c = RCREG1;
237
238     if (c == '<') {
239         rx_idx = 0;
240     }
241
242     if (rx_idx < 48) {
243         rx_buf[rx_idx++] = c;
244     }
245
246     if (c == '>') {
247         if (rx_idx >= 5 && rx_buf[0] == '<') {
248             nxt = (unsigned char)((cmd_qtail + 1) & (CMD_QSIZE - 1));
249             if (nxt != cmd_qhead) {
250                 n = rx_idx;
251                 if (n > (CMD_QLEN - 2)) n = CMD_QLEN - 2;
252                 for (j = 0; j < n; j++)
253                     cmd_queue[cmd_qtail][j] = rx_buf[j];
254                 cmd_queue[cmd_qtail][n] = '\0';
255                 cmd_qtail = nxt;
256             }
257         }
258         rx_idx = 0;
259     }
260 }
261
262 // Fonctions bas niveau pour l'EEPROM I2C
263 void eep_write_byte(unsigned char addr, unsigned char val) {
264     I2C1_Start();
265     I2C1_Wr(EEPROM_W);
266     I2C1_Wr(addr);
267     I2C1_Wr(val);
268     I2C1_Stop();
269     Delay_ms(10);
270 }
271
272 unsigned char eep_read_byte(unsigned char addr) {
273     unsigned char val;
274     I2C1_Start();
275     I2C1_Wr(EEPROM_W);
276     I2C1_Wr(addr);
277     I2C1_Repeated_Start();
278     I2C1_Wr(EEPROM_R);
279     val = I2C1_Rd(0);
280     I2C1_Stop();
281     return val;
282 }
283
284 void eep_write_word(unsigned char addr, unsigned int val) {
285     eep_write_byte(addr, (unsigned char)(val >> 8));
286     eep_write_byte(addr + 1, (unsigned char)(val & 0xFF));
287 }
288
289

```

```

290 unsigned int eep_read_word(unsigned char addr) {
291     unsigned int hi = (unsigned int)eep_read_byte(addr);
292     unsigned int lo = (unsigned int)eep_read_byte(addr + 1);
293     return (hi << 8) | lo;
294 }
295
296 // Mise a jour de la limite haute de la PWM
297 void recalcul_pwm_max() {
298     pwm_max_eff = (unsigned char)((unsigned int)vitesse_max_pc * 255 / 100);
299     if (pwm_max_eff < (unsigned char)(PWM_MIN + 20))
300         pwm_max_eff = (unsigned char)(PWM_MIN + 20);
301 }
302
303 // Recorupere et retire une commande de la file d'attente UART
304 unsigned char pop_cmd(char *dest) {
305     unsigned char i;
306     if (cmd_qhead == cmd_qtail) return 0;
307     GIE_bit = 0;
308     for (i = 0; i < (CMD_QLEN - 1); i++)
309         dest[i] = cmd_queue[cmd_qhead][i];
310     dest[CMD_QLEN - 1] = '\0';
311     cmd_qhead = (unsigned char)((cmd_qhead + 1) & (CMD_QSIZE - 1));
312     GIE_bit = 1;
313     return 1;
314 }
315
316 // Lecture des donnees utilisateur stockees en memoire non volatile
317 void eeprom_charger() {
318     unsigned int stored_traj;
319     unsigned char stored_vit;
320
321     stored_traj = eep_read_word(0x00);
322     last_etage = eep_read_byte(0x02);
323     stored_vit = eep_read_byte(0x03);
324
325     if (stored_traj == 0xFFFF) {
326         nb_trajets = 0;
327         eep_write_word(0x00, 0);
328     } else {
329         nb_trajets = stored_traj;
330     }
331
332     if (last_etage == 0xFF) {
333         last_etage = 0;
334         eep_write_byte(0x02, 0);
335     }
336
337     if (stored_vit == 0xFF || stored_vit == 0) {
338         vitesse_eeprom = 100;
339         vitesse_max_pc = 100;
340         eep_write_byte(0x03, 100);
341     } else {
342         vitesse_eeprom = stored_vit;
343         vitesse_max_pc = stored_vit;
344     }
345
346     poids_max = POIDS_MAX_DEF;
347     seuil_surge = POIDS_MAX_DEF;
348     surcharge_active = 0;

```

```

349     etat_surcharge    = 0;
350
351     recalc_pwm_max();
352 }
353
354 // Enregistre les donnees du trajet actuel en memoire
355 void eeeprom_sauver_trajet() {
356     nb_session++;
357     nb_trajets++;
358     last_etage = etage_actuel;
359     eep_write_word(0x00, nb_trajets);
360     eep_write_byte(0x02, last_etage);
361 }
362
363 // Reset de la memoire EEPROM
364 void eeeprom_reset() {
365     nb_trajets = 0;
366     last_etage = 0;
367     vitesse_eeeprom = 100;
368     vitesse_max_pc = 100;
369     recalc_pwm_max();
370
371     eep_write_byte(0x00, 0x00);
372     eep_write_byte(0x01, 0x00);
373     eep_write_byte(0x02, 0x00);
374     eep_write_byte(0x03, 100);
375     eep_write_byte(0x04, 0x00);
376     eep_write_byte(0x05, 0x00);
377 }
378
379 // Calcul et application du rapport cyclique pour le moteur
380 void set_pwm(unsigned char duty) {
381     PWM1_Set_Duty(duty);
382
383     if (pwm_max_eff > 0) {
384         if (duty >= pwm_max_eff)
385             pwm_actuel = vitesse_max_pc;
386         else
387             pwm_actuel = (unsigned char)((unsigned long)vitesse_max_pc * duty /
388 pwm_max_eff);
389     } else {
390         pwm_actuel = 0;
391     }
392 }
393
394 // Envoi de l'etat complet de la machine par liaison serie
395 void uart_send_data() {
396     char trame[90];
397     unsigned char dir_n, prt_n;
398
399     if (suppress_data_count > 0) {
400         suppress_data_count--;
401         return;
402     }
403
404     if (direction == 'U') dir_n = 1;
405     else if (direction == 'D') dir_n = 2;
406     else dir_n = 0;
407

```

```

408     prt_n = mode_auto ? (ir_porte ? 1 : 0) : (porte_cmd ? 1 : 0);
409
410     sprintf(trame,
411         "<DATA,ET:%d,DIR:%d,PT:%d,PRT:%d,AL:%d,URG:%d,NB:%d,PWM:%d,TPS:%d>\r\n",
412         (int)etage_actuel, (int)dir_n, (int)poids_kg,
413         (int)prt_n, (int)al_active, (int)urg_active,
414         (int)nb_session, (int)pwm_actuel, (int)temps_trajet);
415
416     UART1_Write_Text(trame);
417 }
418
419 // Envoi des donnees de l'EEPROM par liaison serie
420 void uart_send_eeprom() {
421     char trame[60];
422     suppress_data_count = 1;
423     sprintf(trame,
424         "<EEP,TRAJ:%d,LAST:%d,VITESSE:%d>\r\n",
425         (int)nb_trajets, (int)last_etage, (int)vitesse_eeprom);
426     UART1_Write_Text(trame);
427 }
428
429 // Retours d'acquittement UART simple
430 void uart_ack_ok() { UART1_Write_Text("<ACK,OK>\r\n"); }
431 void uart_ack_err() { UART1_Write_Text("<ACK,ERR>\r\n"); }
432
433 // Moyenne glissante sur 8 valeurs ADC pour obtenir la charge cabine
434 void lire_capteurs() {
435     unsigned long somme = 0;
436     unsigned int raw_adc;
437     unsigned char n;
438
439     ir_porte = PORTA.F0;
440
441     ADC_Read(1);
442     for (n = 0; n < 8; n++) {
443         somme += ADC_Read(1);
444     }
445     raw_adc = (unsigned int)(somme >> 3);
446
447     if (raw_adc > POT_ADC_MAX) raw_adc = POT_ADC_MAX;
448     poids_kg = (unsigned int)((raw_adc * 900UL) / POT_ADC_MAX);
449 }
450
451 // Mise a jour du flag de surcharge
452 void maj_surcharge(unsigned char force_transition) {
453     surcharge_active = (poids_kg >= seuil_surge) ? 1 : 0;
454     if (force_transition) {
455         etat_surcharge = 0xFF;
456     }
457     gerer_leds();
458 }
459
460 // Mise a jour visuelle des indicateurs lumineux
461 void gerer_leds() {
462     if (mode_auto) LED1 = ir_porte ? 1 : 0;
463     else LED1 = porte_cmd ? 1 : 0;
464
465     LED2 = (surcharge_active || urg_active || al_active) ? 1 : 0;
466 }

```

```

467 // Convertit le statut de la porte en caractere lisible
468 char etat_porte_char() {
469     return (mode_auto ? ir_porte : porte_cmd) ? 'O' : 'F';
470 }
471
472 // Generation de la chaîne de texte liee a la surcharge pour la ligne 2 du LCD
473 void lcd_build_surcharge_12(char *buf) {
474     unsigned int v = poids_max;
475
476     buf[0] = ' ';
477     buf[1] = ' ';
478     buf[2] = 'M';
479     buf[3] = 'A';
480     buf[4] = 'X';
481     buf[5] = ':';
482     buf[6] = (v >= 100) ? ((char)('0' + (v / 100))) : ' ';
483     buf[7] = (v >= 10) ? ((char)('0' + ((v / 10) % 10))) : ' ';
484     buf[8] = (char)('0' + (v % 10));
485     buf[9] = ' ';
486     buf[10] = 'k';
487     buf[11] = 'g';
488     buf[12] = ' ';
489     buf[13] = ' ';
490     buf[14] = ' ';
491     buf[15] = ' ';
492     buf[16] = '\0';
493 }
494
495 // Gestion des informations de l'ecran LCD principal au repos
496 void afficher_lcd() {
497     char dir_str[3];
498     char mode_str[5];
499     char surcharge_12[17];
500
501     if (position_inconnue) {
502         Lcd_Out(1, 1, "POS INCONNUE! ");
503         Lcd_Out(2, 1, "Replacer manuel ");
504         return;
505     }
506
507     if (!urg_active && !al_active && surcharge_active) {
508         lcd_build_surcharge_12(surcharge_12);
509         Lcd_Out(1, 1, "SURCHARGE! ");
510         Lcd_Out(2, 1, surcharge_12);
511         return;
512     }
513
514     if (mode_auto) {
515         mode_str[0]='A'; mode_str[1]='u'; mode_str[2]='t'; mode_str[3]='o';
516         mode_str[4]='\0';
517     } else {
518         mode_str[0]='M'; mode_str[1]='a'; mode_str[2]='n'; mode_str[3]='u';
519         mode_str[4]='\0';
520     }
521
522     if (en_mouvement) {
523         if (direction == 'U') { dir_str[0]='U'; dir_str[1]='P'; dir_str[2]='\0';
524         }
525     }

```

```

526         else                                { dir_str[0]='D'; dir_str[1]='N'; dir_str[2]='\0';
527     }
528
529     sprintf(l1, "ET:%u->%u %s %s ",
530         (unsigned)etage_actuel, (unsigned)etage_cible, dir_str, mode_str);
531 } else {
532     sprintf(l1, "ET:%u STOP %s ",
533         (unsigned)etage_actuel, mode_str);
534 }
535
536 Lcd_Out(1, 1, l1);
537 sprintf(l2, "P:%3dkg IR:%c ", (int)poids_kg, etat_porte_char());
538 Lcd_Out(2, 1, l2);
539 }
540
541 // Transition propre de l'affichage LCD
542 void lcd_transition() {
543     Lcd_Cmd(_LCD_CLEAR);
544     afficher_lcd();
545 }
546
547 // Gestion de l'affichage specifique lors des transits cabine
548 void lcd_update_transit() {
549     char dir_str[3];
550     char mode_str[5];
551
552     if (direction == 'U') { dir_str[0]='U'; dir_str[1]='P'; dir_str[2]='\0'; }
553     else                    { dir_str[0]='D'; dir_str[1]='N'; dir_str[2]='\0'; }
554
555     if (mode_auto) {
556         mode_str[0]='A'; mode_str[1]='u'; mode_str[2]='t'; mode_str[3]='o';
557     mode_str[4]='\0';
558     } else {
559         mode_str[0]='M'; mode_str[1]='a'; mode_str[2]='n'; mode_str[3]='u';
560     mode_str[4]='\0';
561     }
562
563 #ifndef SIMULATION_PROTEUS
564     PWM1_Set_Duty(0);
565     Delay_ms(MS_PWM_PAUSE);
566 #endif
567
568     sprintf(l1, "ET:%u->%u %s %s ",
569         (unsigned)etage_actuel, (unsigned)etage_cible, dir_str, mode_str);
570     Lcd_Out(1, 1, l1);
571
572     sprintf(l2, "P:%3dkg IR:%c ", (int)poids_kg, etat_porte_char());
573     Lcd_Out(2, 1, l2);
574
575 #ifndef SIMULATION_PROTEUS
576     PWM1_Set_Duty(pwm_max_eff);
577 #endif
578 }
579
580 // Applique et met a jour la configuration de charge maximale
581 void appliquer_pmax(unsigned int val) {
582     if (val < 1)    val = 1;
583     if (val > 999) val = 999;
584 }

```

```

585     poids_max = val;
586     seuil_surge = val;
587
588     lire_capteurs();
589     maj_surcharge(1);
590
591     uart_ack_ok();
592     uart_send_data();
593 }
594
595 // Applique et met a jour la vitesse cible
596 void appliquer_spd(unsigned char val) {
597     if (val > 100) val = 100;
598     if (val < 10)  val = 10;
599
600     vitesse_max_pc = val;
601     vitesse_eeprom = val;
602     recalc_pwm_max();
603     eep_write_byte(0x03, vitesse_eeprom);
604
605     if (moteur_actif) set_pwm(pwm_max_eff);
606
607     suppress_data_count = 1;
608     uart_ack_ok();
609     uart_send_data();
610 }
611
612 // Attente temporisee non bloquante qui surveille la securite et les ordres serie
613 void attendre_ms(unsigned int ms) {
614     unsigned int elapsed = 0;
615     unsigned char b;
616     char local_cmd[50];
617     char *pp;
618     unsigned int pv;
619     unsigned char sv;
620     unsigned char nc;
621
622     while (elapsed < ms) {
623
624         if (BP_URGENCE && !urg_active) {
625             Delay_ms(20);
626             if (BP_URGENCE) {
627                 MOTEUR_ARRETER();
628                 urg_active = 1;
629                 LED2 = 1;
630                 urgence_flag = 1;
631             }
632         }
633
634         if (urgence_flag || stop_demande) return;
635
636         // Analyse immediate des trames en attente dans la file
637         if (pop_cmd(local_cmd)) {
638
639             if (strstr(local_cmd, "CMD,STOP")) {
640                 MOTEUR_ARRETER();
641                 urg_active = 1;
642                 direction = 'S';
643                 en_mouvement = 0;

```

```

644         LED2 = 1;
645         urgence_flag = 1;
646         uart_ack_ok();
647
648     } else if (strstr(local_cmd, "MOT:STP") || strstr(local_cmd,
649 "MOT:STOP")) {
650
651         MOTEUR_ARRETER();
652         stop_demande = 1;
653         uart_ack_ok();
654
655     } else if (strstr(local_cmd, "AL:ON")) {
656         al_active = 1;
657         al_flag = 1;
658         LED2 = 1;
659         uart_ack_ok();
660
661     } else if ((pp = strstr(local_cmd, "CALL:")) != 0) {
662         nc = (unsigned char) (*pp + 5) - '0';
663         if (nc < NB_ETAGES && !al_active && !surcharge_active) {
664             if (direction == 'U' && nc > etage_actuel) req[nc] = 1;
665             else if (direction == 'D' && nc < etage_actuel) req[nc] = 1;
666             uart_ack_ok();
667         } else {
668             uart_ack_err();
669         }
670
671     } else if ((pp = strstr(local_cmd, "PMAX:")) != 0) {
672         pv = 0;
673         pp += 5;
674         while (*pp >= '0' && *pp <= '9') {
675             pv = pv * 10 + (unsigned int) (*pp - '0');
676             pp++;
677         }
678         appliquer_pmax(pv);
679
680     } else if ((pp = strstr(local_cmd, "SPD:")) != 0) {
681         sv = 0;
682         pp += 4;
683         while (*pp >= '0' && *pp <= '9') {
684             sv = sv * 10 + (unsigned char) (*pp - '0');
685             pp++;
686         }
687         appliquer_spd(sv);
688
689     } else if (strstr(local_cmd, "GET:EEP")) {
690         uart_send_eeprom();
691
692     } else if (strstr(local_cmd, "EEP:RST") || strstr(local_cmd,
693 "RST:EEP")) {
694         eeprom_reset();
695         uart_ack_ok();
696         uart_send_eeprom();
697     }
698 }
699
700 if (urgence_flag || stop_demande) return;
701
702 // Lecture immediate des boutons poussoirs d'etages (PORTD)

```

```

703     for (b = 0; b < NB_ETAGES; b++) {
704         if (PORTD & (1 << b)) {
705             if (direction == 'U' && b > etage_actuel) req[b] = 1;
706             else if (direction == 'D' && b < etage_actuel) req[b] = 1;
707         }
708     }
709
710     if (timer0_flag) {
711         timer0_flag = 0;
712         lire_capteurs();
713         maj_surcharge(0);
714         uart_send_data();
715
716         if (moteur_actif) lcd_update_transit();
717         else               afficher_lcd();
718     }
719
720     Delay_ms(MS_LOOP_STEP);
721     elapsed += MS_LOOP_STEP;
722 }
723
724
725 // Accelération graduelle du moteur
726 void rampe_accel() {
727     unsigned char i;
728     unsigned int  pwm;
729
730     for (i = 0; i <= PWM_PALIERES; i++) {
731         if (BP_URGENCE && !urg_active) {
732             Delay_ms(20);
733             if (BP_URGENCE) {
734                 MOTEUR_ARRETER();
735                 urg_active = 1;
736                 LED2 = 1;
737                 urgence_flag = 1;
738             }
739         }
740
741         if (urgence_flag || stop_demande) {
742             MOTEUR_ARRETER();
743             return;
744         }
745
746         pwm = PWM_MIN + ((unsigned int) (pwm_max_eff - PWM_MIN) * i) /
PWM_PALIERES;
747         set_pwm((unsigned char) pwm);
748         uart_send_data();
749         Delay_ms(MS_PALIER);
750     }
751 }
752
753
754 // Deceleration progressive du moteur avant l'arret
755 void rampe_decel() {
756     unsigned char i;
757     unsigned int  pwm;
758
759     for (i = PWM_PALIERES; i > 0; i--) {
760         if (BP_URGENCE && !urg_active) {
761             Delay_ms(20);

```

```

762         if (BP_URGENCE) {
763             MOTEUR_ARRETER();
764             urg_active = 1;
765             LED2 = 1;
766             urgence_flag = 1;
767         }
768     }
769
770     if (urgence_flag || stop_demande) {
771         MOTEUR_ARRETER();
772         return;
773     }
774
775     pwm = PWM_MIN + ((unsigned int) (pwm_max_eff - PWM_MIN) * (i - 1)) /
PWM_PALIERS;
776     set_pwm((unsigned char)pwm);
777     uart_send_data();
778     Delay_ms(MS_PALIER);
779 }
780
781 MOTEUR_ARRETER();
782 }
783
784 // Initialise l'activation moteur et lance la rampe d'acceleration
785 void demarrer_moteur(char sens) {
786     if (sens == 'U') MOTEUR_MONTER();
787     else MOTEUR_DESCENDRE();
788
789     set_pwm(PWM_MIN);
790     PWM1_Start();
791     rampe_accel();
792 }
793
794 // Lecture brute des entrees logiques des appels d'etages
795 void scanner_req() {
796     if (PORTD.F0) req[0] = 1;
797     if (PORTD.F1) req[1] = 1;
798     if (PORTD.F2) req[2] = 1;
799     if (PORTD.F3) req[3] = 1;
800 }
801
802 // Vide le tableau complet des demandes
803 void vider_req() {
804     unsigned char i;
805     for (i = 0; i < NB_ETAGES; i++) req[i] = 0;
806 }
807
808 // Choisit l'etage prioritaire a desservir selon le sens actuel
809 unsigned char prochain_req() {
810     unsigned char i, nearest;
811     unsigned int d, min_d;
812
813     req[etage_actuel] = 0;
814
815     if (direction == 'U') {
816         for (i = etage_actuel + 1; i < NB_ETAGES; i++)
817             if (req[i]) { req[i] = 0; return i; }
818
819         for (i = 0; i < etage_actuel; i++)

```

```

821         if (req[i]) { req[i] = 0; return i; }
822
823     } else if (direction == 'D') {
824         for (i = etage_actuel; i > 0; i--)
825             if (req[i - 1]) { req[i - 1] = 0; return i - 1; }
826
827         for (i = etage_actuel + 1; i < NB_ETAGES; i++)
828             if (req[i]) { req[i] = 0; return i; }
829
830     } else {
831         nearest = 0xFF;
832         min_d = 10;
833
834         for (i = 0; i < NB_ETAGES; i++) {
835             if (!req[i]) continue;
836
837             d = (i >= etage_actuel) ? (unsigned int)(i - etage_actuel)
838                                     : (unsigned int)(etage_actuel - i);
839
840             if (d < min_d) {
841                 min_d = d;
842                 nearest = i;
843             }
844         }
845
846         if (nearest != 0xFF) {
847             req[nearest] = 0;
848             return nearest;
849         }
850     }
851
852     return 0xFF;
853 }
854
855 // Logique globale du cycle de déplacement entre les etages
856 void deplacer_vers(unsigned char cible) {
857     char sens;
858     unsigned char nb_et, i, est_dernier;
859     unsigned int temps_debut;
860
861     if (cible == etage_actuel || urgence_flag) return;
862
863     stop_demande = 0;
864
865     sens = (cible > etage_actuel) ? 'U' : 'D';
866     direction = sens;
867     etage_cible = cible;
868     en_mouvement = 1;
869
870     lire_capteurs();
871     maj_surcharge(1);
872     lcd_transition();
873     uart_send_data();
874     temps_debut = timer0_count;
875
876     attendre_ms(MS_FERMETURE);
877     if (urgence_flag || stop_demande) goto fin_deplacement;
878
879

```

```

880     nb_et = (cible > etage_actuel) ? (cible - etage_actuel) : (etage_actuel -
881     cible);
882
883     demarrer_moteur(sens);
884     lcd_update_transit();
885
886     for (i = 0; i < nb_et; i++) {
887         if (urgence_flag || stop_demande) goto fin_deplacement;
888
889         est_dernier = (i == nb_et - 1);
890         set_pwm(pwm_max_eff);
891
892         if (!est_dernier) {
893             entre_etages = 1;
894             attendre_ms(MS_CROISIERE);
895             if (urgence_flag || stop_demande) goto fin_deplacement;
896             entre_etages = 0;
897
898             if (sens == 'U') etage_actuel++;
899             else etage_actuel--;
900
901             lire_capteurs();
902             maj_surcharge(0);
903             lcd_update_transit();
904             uart_send_data();
905
906             if (req[etage_actuel]) {
907                 req[etage_actuel] = 0;
908                 rampe_decel();
909                 if (urgence_flag || stop_demande) goto fin_deplacement;
910
911                 en_mouvement = 0;
912                 direction = 'S';
913                 etage_cible = etage_actuel;
914                 lire_capteurs();
915                 maj_surcharge(1);
916                 lcd_transition();
917                 uart_send_data();
918
919                 attendre_ms(MS_ARRET_INTERMED);
920                 if (urgence_flag || stop_demande) goto fin_deplacement;
921
922                 direction = sens;
923                 etage_cible = cible;
924                 en_mouvement = 1;
925                 lcd_transition();
926                 attendre_ms(MS_FERMETURE);
927                 if (urgence_flag || stop_demande) goto fin_deplacement;
928                 demarrer_moteur(sens);
929                 lcd_update_transit();
930             }
931
932         } else {
933             entre_etages = 1;
934             if (nb_et == 1) attendre_ms(MS_CROISIERE_1ET);
935             else attendre_ms(MS_CROISIERE);
936
937             if (urgence_flag || stop_demande) goto fin_deplacement;
938             entre_etages = 0;

```

```

939         rampe_decel();
940         if (urgence_flag || stop_demande) {
941             position_inconnue = 1;
942             goto fin_deplacement;
943         }
944
945         if (sens == 'U') etage_actuel++;
946         else etage_actuel--;
947     }
948 }
949
950
951 fin_deplacement:
952     MOTEUR_ARRETER();
953
954     if ((urgence_flag || stop_demande) && entre_etages) {
955         position_inconnue = 1;
956         entre_etages = 0;
957     }
958
959     temps_trajet = timer0_count - temps_debut;
960     en_mouvement = 0;
961     direction = 'S';
962     etage_cible = etage_actuel;
963     etat_surcharge = 0xFF;
964
965     if (!urgence_flag && !stop_demande) {
966         position_inconnue = 0;
967     }
968
969     lire_capteurs();
970     maj_surcharge(1);
971     lcd_transition();
972     uart_send_data();
973
974     if (!urgence_flag && !stop_demande) {
975         eeprom_sauver_trajet();
976         attendre_ms(MS_OUVERTURE);
977     }
978
979     stop_demande = 0;
980 }
981
982 // Decodage des commandes textes recues via la communication UART
983 void parser_cmd(char *buf) {
984     char *p;
985     unsigned char cible;
986     unsigned int pv;
987     unsigned char sv;
988
989     p = strstr(buf, "CALL:");
990     if (p) {
991         cible = (unsigned char) (*(p + 5) - '0');
992         if (cible < NB_ETAGES && mode_auto && !al_active && !surcharge_active) {
993             req[cible] = 1;
994             uart_ack_ok();
995         } else {
996             uart_ack_err();
997         }
998     }

```

```

998     return;
999 }
1000
1001 if (strstr(buf, "CMD,STOP")) {
1002     MOTEUR_ARRETER();
1003     urg_active = 1;
1004     direction = 'S';
1005     en_mouvement = 0;
1006     LED2 = 1;
1007     urgence_flag = 1;
1008     uart_ack_ok();
1009     uart_send_data();
1010     return;
1011 }
1012
1013 if (strstr(buf, "CMD,ACK")) {
1014     if (BP_URGENCE) {
1015         uart_ack_err();
1016     } else if (urg_active) {
1017         urg_active = 0;
1018         al_active = 0;
1019         urgence_flag = 0;
1020         position_inconnue = 0;
1021         direction = 'S';
1022         en_mouvement = 0;
1023         etat_surcharge = 0xFF;
1024         lire_capteurs();
1025         maj_surcharge(1);
1026         lcd_transition();
1027         uart_ack_ok();
1028         uart_send_data();
1029     } else if (surcharge_active) {
1030         uart_ack_err();
1031     } else {
1032         uart_ack_ok();
1033     }
1034     return;
1035 }
1036
1037 if (strstr(buf, "MODE:AUTO")) {
1038     mode_auto = 1;
1039     direction = 'S';
1040     en_mouvement = 0;
1041     etat_surcharge = 0xFF;
1042     lire_capteurs();
1043     maj_surcharge(1);
1044     lcd_transition();
1045     uart_ack_ok();
1046     uart_send_data();
1047     return;
1048 }
1049
1050 if (strstr(buf, "MODE:MAN")) {
1051     mode_auto = 0;
1052     direction = 'S';
1053     en_mouvement = 0;
1054     MOTEUR_ARRETER();
1055     etat_surcharge = 0xFF;
1056     lire_capteurs();

```

```

1057     maj_surcharge(1);
1058     lcd_transition();
1059     uart_ack_ok();
1060     uart_send_data();
1061     return;
1062 }
1063
1064 if (strstr(buf, "GET:EEP")) {
1065     uart_send_eeprom();
1066     return;
1067 }
1068
1069 if (strstr(buf, "EEP:RST") || strstr(buf, "RST:EEP")) {
1070     eeprom_reset();
1071     uart_ack_ok();
1072     uart_send_eeprom();
1073     return;
1074 }
1075
1076 {
1077     char *pP = strstr(buf, "PMAx:");
1078     char *pS = strstr(buf, "SPD:");
1079     if (pP && pS) {
1080         pv = 0;
1081         pP += 5;
1082         while (*pP >= '0' && *pP <= '9') {
1083             pv = pv * 10 + (unsigned int)(*pP - '0');
1084             pP++;
1085         }
1086         sv = 0;
1087         pS += 4;
1088         while (*pS >= '0' && *pS <= '9') {
1089             sv = sv * 10 + (unsigned char)(*pS - '0');
1090             pS++;
1091         }
1092
1093         if (pv < 1)    pv = 1;
1094         if (pv > 999) pv = 999;
1095         poids_max    = pv;
1096         seuil_surge  = pv;
1097
1098         if (sv > 100) sv = 100;
1099         if (sv < 10)  sv = 10;
1100         vitesse_max_pc = sv;
1101         vitesse_eeprom = sv;
1102         recalc_pwm_max();
1103         eep_write_byte(0x03, vitesse_eeprom);
1104         if (moteur_actif) set_pwm(pwm_max_eff);
1105
1106         lire_capteurs();
1107         maj_surcharge(1);
1108         uart_ack_ok();
1109         uart_send_data();
1110         return;
1111     }
1112 }
1113
1114 p = strstr(buf, "PMAx:");
1115 if (p) {

```

```

1116     pv = 0;
1117     p += 5;
1118     while (*p >= '0' && *p <= '9') {
1119         pv = pv * 10 + (unsigned int) (*p - '0');
1120         p++;
1121     }
1122     appliquer_pmax(pv);
1123     return;
1124 }
1125
1126 p = strstr(buf, "SPD:");
1127 if (p) {
1128     sv = 0;
1129     p += 4;
1130     while (*p >= '0' && *p <= '9') {
1131         sv = sv * 10 + (unsigned char) (*p - '0');
1132         p++;
1133     }
1134     appliquer_spd(sv);
1135     return;
1136 }
1137
1138 if (strstr(buf, "DOOR:O")) {
1139     if (!mode_auto) {
1140         porte_cmd = 1;
1141         gerer_leds();
1142         if (!moteur_actif) lcd_transition();
1143         uart_ack_ok();
1144         uart_send_data();
1145     } else {
1146         uart_ack_err();
1147     }
1148     return;
1149 }
1150
1151 if (strstr(buf, "DOOR:F")) {
1152     if (!mode_auto) {
1153         porte_cmd = 0;
1154         gerer_leds();
1155         if (!moteur_actif) lcd_transition();
1156         uart_ack_ok();
1157         uart_send_data();
1158     } else {
1159         uart_ack_err();
1160     }
1161     return;
1162 }
1163
1164 if (strstr(buf, "MOT:UP")) {
1165     if (!mode_auto && !urg_active && !al_active && !surcharge_active) {
1166         if (moteur_actif || en_mouvement) {
1167             uart_ack_err();
1168         } else if (etage_actuel >= NB_ETAGES - 1) {
1169             uart_ack_err();
1170         } else {
1171             uart_ack_ok();
1172             deplacer_vers(etage_actuel + 1);
1173         }
1174     } else {

```

```

1175         uart_ack_err();
1176     }
1177     return;
1178 }
1179
1180 if (strstr(buf, "MOT:DWN")) {
1181     if (!mode_auto && !urg_active && !al_active && !surcharge_active) {
1182         if (moteur_actif || en_mouvement) {
1183             uart_ack_err();
1184         } else if (etage_actuel == 0) {
1185             uart_ack_err();
1186         } else {
1187             uart_ack_ok();
1188             deplacer_vers(etage_actuel - 1);
1189         }
1190     } else {
1191         uart_ack_err();
1192     }
1193     return;
1194 }
1195
1196 if (strstr(buf, "MOT:STP") || strstr(buf, "MOT:STOP")) {
1197     if (!mode_auto) {
1198         MOTEUR_ARRETER();
1199         direction = 'S';
1200         en_mouvement = 0;
1201         etat_surcharge = 0xFF;
1202         lire_capteurs();
1203         maj_surcharge(1);
1204         lcd_transition();
1205         uart_ack_ok();
1206         uart_send_data();
1207     } else {
1208         uart_ack_err();
1209     }
1210     return;
1211 }
1212
1213 if (strstr(buf, "AL:ON")) {
1214     al_active = 1;
1215     al_flag = 1;
1216     LED2 = 1;
1217     en_mouvement = 0;
1218     MOTEUR_ARRETER();
1219     uart_ack_ok();
1220     uart_send_data();
1221     return;
1222 }
1223
1224 if (strstr(buf, "RST:AL")) {
1225     if (!urg_active) {
1226         al_active = 0;
1227         al_flag = 0;
1228         etat_surcharge = 0xFF;
1229         lire_capteurs();
1230         maj_surcharge(1);
1231         lcd_transition();
1232         uart_ack_ok();
1233         uart_send_data();

```

```

1234         } else {
1235             uart_ack_err();
1236         }
1237         return;
1238     }
1239
1240     uart_ack_err();
1241 }
1242
1243 // Fonction d'entree de l'application
1244 void main() {
1245     unsigned char prochain;
1246     char          cmd[50];
1247     unsigned char surge_now;
1248     unsigned char dummy_pb;
1249
1250     // Config PWM
1251     PWM1_Init(5000);
1252     PWM1_Set_Duty(0);
1253     PWM1_Start();
1254
1255     // Config I/O PORTA
1256     ANSELA = 0x02;
1257     TRISA0_bit = 1;
1258     TRISA1_bit = 1;
1259     TRISA2_bit = 0;
1260     TRISA3_bit = 0;
1261     LATA2_bit = 0;
1262     LATA3_bit = 0;
1263
1264     // Config I/O PORTB
1265     ANSELB = 0x00;
1266     TRISB6_bit = 1;
1267     TRISB7_bit = 1;
1268     INTCON2.RBPU = 1;
1269
1270     // Config I/O PORTC
1271     ANSELB = 0x00;
1272     TRISC0_bit = 0;
1273     TRISC1_bit = 0;
1274     TRISC2_bit = 0;
1275     TRISC3_bit = 0;
1276     TRISC4_bit = 1;
1277     TRISC6_bit = 0;
1278     TRISC7_bit = 1;
1279     LATC0_bit = 0;
1280     LATC1_bit = 0;
1281
1282     // Config I/O PORTD
1283     ANSELD = 0x00;
1284     TRISD = 0xFF;
1285
1286     // Config ADC
1287     ADC_Init();
1288     ANSELA = 0x02;
1289     TRISA0_bit = 1;
1290     TRISA1_bit = 1;
1291
1292     // Config UART (9600 bauds)

```

```

1293     UART1_Init(9600);
1294     Delay_ms(100);
1295
1296     // Config Bus I2C
1297     I2C1_Init(100000);
1298     Delay_ms(10);
1299
1300     // Initialisation memoire et Timer0
1301     eeprom_charger();
1302
1303     T0CON = 0x07;
1304     TMR0H = T0_RELOAD_H;
1305     TMR0L = T0_RELOAD_L;
1306     TMR0IF_bit = 0;
1307     TMR0IE_bit = 1;
1308
1309     dummy_pb = PORTB;
1310     RBIF_bit = 0;
1311     RBIE_bit = 1;
1312
1313     // Activation globale des interruptions
1314     RC1IE_bit = 1;
1315     PEIE_bit = 1;
1316     GIE_bit = 1;
1317     T0CON = 0x87;
1318
1319     // Message d'accueil LCD
1320     Lcd_Init();
1321     Lcd_Cmd(_LCD_CLEAR);
1322     Lcd_Cmd(_LCD_CURSOR_OFF);
1323     Lcd_Out(1, 1, " ASCENSEUR 4ET ");
1324     Lcd_Out(2, 1, " Pret - ET:0 ");
1325     Delay_ms(1500);
1326     Lcd_Cmd(_LCD_CLEAR);
1327
1328
1329     UART1_Write_Text("<DATA,ET:0,DIR:0,PT:0,PRT:0,AL:0,URG:0,NB:0,PWM:0,TPS:0>\r\n");
1330
1331     lire_capteurs();
1332     maj_surcharge(1);
1333     lcd_transition();
1334
1335     // Boucle de scrutation principale
1336     while (1) {
1337
1338         if (BP_URGENCE && !urg_active) {
1339             Delay_ms(20);
1340             if (BP_URGENCE) {
1341                 MOTEUR_ARRETER();
1342                 direction = 'S';
1343                 en_mouvement = 0;
1344                 urg_active = 1;
1345                 LED2 = 1;
1346                 urgence_flag = 1;
1347             }
1348         }
1349
1350         // Cycle d'arret d'urgence et attente d'acquittement
1351         if (urgence_flag) {

```

```

1352     urgence_flag = 0;
1353     etat_surcharge = 0;
1354     MOTEUR_ARRETER();
1355     en_mouvement = 0;
1356     direction = 'S';
1357     vider_req();
1358
1359     Lcd_Cmd(_LCD_CLEAR);
1360     Lcd_Out(1, 1, " ARRET URGENCE ");
1361     if (position_inconnue) Lcd_Out(2, 1, "POS?-ACQ:BP/PC ");
1362     else Lcd_Out(2, 1, "ACQ : BP ou PC ");
1363
1364     lire_capteurs();
1365     maj_surcharge(0);
1366     uart_send_data();
1367
1368     {
1369         unsigned char acq_recu = 0;
1370         while (acq_recu == 0) {
1371             Delay_ms(MS_LOOP_STEP);
1372             if (timer0_flag) {
1373                 timer0_flag = 0;
1374                 uart_send_data();
1375             }
1376
1377             if (BP_ACQ && !BP_URGENCE) {
1378                 Delay_ms(50);
1379                 acq_recu = 1;
1380             }
1381
1382             if (pop_cmd(cmd)) {
1383                 if (strstr(cmd, "CMD,ACK") && !BP_URGENCE)
1384                     acq_recu = 1;
1385             }
1386         }
1387         Delay_ms(200);
1388     }
1389
1390     urg_active = 0;
1391     al_active = 0;
1392     al_flag = 0;
1393     position_inconnue = 0;
1394     direction = 'S';
1395     en_mouvement = 0;
1396     etat_surcharge = 0xFF;
1397
1398     lire_capteurs();
1399     maj_surcharge(1);
1400     lcd_transition();
1401     uart_send_data();
1402 }
1403
1404 // Cycle d'alarme active et attente d'acquiescement
1405 if (al_flag) {
1406     al_flag = 0;
1407     vider_req();
1408
1409     Lcd_Cmd(_LCD_CLEAR);
1410     Lcd_Out(1, 1, " ALARME !!! ");

```

```

1411     Lcd_Out(2, 1, "ACQ : BP ou PC ");
1412     uart_send_data();
1413
1414     {
1415         unsigned char acq_recu = 0;
1416         while (acq_recu == 0) {
1417             Delay_ms(MS_LOOP_STEP);
1418             if (timer0_flag) {
1419                 timer0_flag = 0;
1420                 uart_send_data();
1421             }
1422             if (BP_ACQ) {
1423                 Delay_ms(50);
1424                 acq_recu = 1;
1425             }
1426             if (pop_cmd(cmd)) {
1427                 if (strstr(cmd, "RST:AL"))
1428                     acq_recu = 1;
1429             }
1430         }
1431         Delay_ms(200);
1432     }
1433
1434     al_active = 0;
1435     etat_surcharge = 0xFF;
1436     lire_capteurs();
1437     maj_surcharge(1);
1438     lcd_transition();
1439     uart_send_data();
1440 }
1441
1442 if (BP_ALARME && !al_active) {
1443     Delay_ms(20);
1444     if (BP_ALARME) {
1445         al_active = 1;
1446         LED2 = 1;
1447         en_mouvement = 0;
1448         MOTEUR_ARRETER();
1449         vider_req();
1450
1451         Lcd_Cmd(_LCD_CLEAR);
1452         Lcd_Out(1, 1, "ALARME !!! ");
1453         Lcd_Out(2, 1, "ACQ : BP ou PC ");
1454         uart_send_data();
1455
1456         {
1457             unsigned char acq_recu = 0;
1458             while (acq_recu == 0) {
1459                 Delay_ms(MS_LOOP_STEP);
1460                 if (timer0_flag) {
1461                     timer0_flag = 0;
1462                     uart_send_data();
1463                 }
1464                 if (BP_ACQ) {
1465                     Delay_ms(50);
1466                     acq_recu = 1;
1467                 }
1468                 if (pop_cmd(cmd)) {
1469                     if (strstr(cmd, "RST:AL"))

```

```

1470         acq_recu = 1;
1471     }
1472 }
1473     Delay_ms(200);
1474 }
1475
1476     al_active = 0;
1477     al_flag = 0;
1478     etat_surcharge = 0xFF;
1479     lire_capteurs();
1480     maj_surcharge(1);
1481     lcd_transition();
1482     uart_send_data();
1483 }
1484 }
1485
1486 lire_capteurs();
1487 maj_surcharge(0);
1488
1489 while (pop_cmd(cmd)) {
1490     parser_cmd(cmd);
1491 }
1492
1493 gerer_leds();
1494
1495 if (!urg_active && !al_active) {
1496     surge_now = surcharge_active ? 1 : 0;
1497     if (surge_now != etat_surcharge) {
1498         etat_surcharge = surge_now;
1499         if (!moteur_actif) lcd_transition();
1500         uart_send_data();
1501     }
1502 }
1503
1504 if (timer0_flag) {
1505     timer0_flag = 0;
1506     uart_send_data();
1507     if (moteur_actif) lcd_update_transit();
1508     else afficher_lcd();
1509 }
1510
1511 // Routage automatique en fonction des requêtes d'etages recoltees
1512 if (mode_auto && !urg_active && !al_active && !position_inconnue) {
1513     if (!surcharge_active) {
1514         scanner_req();
1515         prochain = prochain_req();
1516
1517         if (prochain != 0xFF) {
1518             lire_capteurs();
1519             maj_surcharge(0);
1520
1521             if (ir_porte == 1) {
1522                 Lcd_Cmd(_LCD_CLEAR);
1523                 Lcd_Out(1, 1, "PORTE OUVERTE! ");
1524                 Lcd_Out(2, 1, "Veuillez fermer ");
1525                 Delay_ms(2000);
1526                 lcd_transition();
1527             }
1528             else if (surcharge_active) {

```

```
        uart_send_data();
    }
    else {
        deplacer_vers(prochain);
        direction = 'S';
        etat_surcharge = 0xFF;
    }
}

}

if (!moteur_actif) afficher_lcd();
Delay_ms(MS_LOOP_STEP);
}
```